

Design Patterns

In This Edition

1	Overview --- What It Is	3
2	Why It Exists	4
3	Why FAANG Cares	4
4	Core Concepts	5
	4.1 1. Program to an Interface, Not an Implementation	5
	4.2 2. Favor Composition Over Inheritance	5
	4.3 3. Encapsulate What Varies	6
	4.4 4. SOLID Recap (the principles patterns enforce)	6
	4.5 5. The Hollywood Principle --- "Don't call us, we'll call you"	6
5	Creational Patterns	7
	5.1 Singleton	7
	5.2 Factory Method	8
	5.3 Abstract Factory	9
	5.4 Builder	11
	5.5 Prototype	11
	5.6 Creational Patterns --- Comparison	12
6	Structural Patterns	13
	6.1 Adapter	13
	6.2 Decorator	14
	6.3 Facade	15
	6.4 Proxy	16
	6.5 Composite	17
	6.6 Bridge	18
	6.7 Flyweight	19
	6.8 Structural Patterns --- Comparison	20
7	Behavioral Patterns	20
	7.1 Strategy	20
	7.2 Observer	21
	7.3 Command	22
	7.4 State	24
	7.5 Template Method	25
	7.6 Iterator	26
	7.7 Chain of Responsibility	27
	7.8 Mediator	28
	7.9 Memento	29
	7.10 Visitor	30
	7.11 Interpreter	31
	7.12 Behavioral Patterns --- Comparison	32
8	Modern / Architectural Patterns	32
	8.1 Dependency Injection / Inversion of Control (IoC)	32
	8.2 MVC / MVP / MVVM	33
	8.3 Repository	34
	8.4 Null Object	34
	8.5 Object Pool	35
9	Which Pattern Do I Use? (Decision Guide)	35
10	Patterns in the Wild & How They Combine	37
11	Anti-Patterns	37
	11.1 God Object (a.k.a. Blob)	37
	11.2 Golden Hammer	38
	11.3 Spaghetti Code	38
	11.4 Poltergeist (a.k.a. Gypsy class)	38
	11.5 Premature Abstraction / Speculative Generality	38
	11.6 Singleton Abuse	38
	11.7 Bonus anti-patterns worth naming	38
12	Architecture / Diagrams	39

14	Real-Life Analogies	40
15	Memory Tricks / Mnemonics	42
15.1	The three families	42
15.2	Creational --- "SABFP"	42
15.3	Structural --- "A Big Cat Drinks From Pool" (ABCDP + Proxy)	42
15.4	Behavioral --- "Smart Os Cause State To Iterate Calmly, Mostly Visiting Interpreters"	42
15.5	Killer distinctions (the ones interviewers probe)	42
16	Common Interview Questions	42
16.1	Q1: What is a design pattern, and when should you NOT use one?	42
16.2	Q2: Explain Strategy vs. State --- they look identical.	43
16.3	Q3: Why is Singleton often called an anti-pattern, and what's the alternative?	43
16.4	Q4: Decorator vs. Proxy vs. Inheritance --- distinguish them.	43
16.5	Q5: Factory Method vs. Abstract Factory?	44
16.6	Q6: How does Observer avoid tight coupling, and what's the classic bug?	44
16.7	Q7: Give a pattern that embodies the Open/Closed Principle and explain how.	44
16.8	Q8: What pattern fits a request/middleware pipeline?	44
16.9	Q9: How would you implement undo/redo, and which patterns?	45
16.10	Q10: When would you choose Composition over Inheritance, and which patterns reflect that?	45
16.11	Q11: How do real frameworks combine multiple patterns? Give a concrete example	45
16.12	Q12: What's an anti-pattern, and how do you fix the God Object?	45
17	Senior-Level Discussion Points	46
18	Typical Mistakes Candidates Make	46
19	How This Connects to Other Topics	47
20	FAANG Interview Tips	47
21	Revision Cheat Sheet	48
21.1	Families	48
21.2	One-liners	48
21.3	Killer distinctions (memorize)	49
21.4	Singleton thread-safe forms (Java)	49
21.5	Anti-patterns → fixes	49
21.6	Golden rules	49

How to use this file: Read top-to-bottom for deep mastery. For last-minute revision, jump to §Common Interview Questions + §Revision Cheat Sheet. Patterns are the working vocabulary of **Low-Level Design (§06)** — internalize each pattern's *intent* and the *force* it relieves, not just the UML box-and-arrow. Every pattern below has: Intent, the concrete Problem it solves, a Structure diagram, real Java/Python code, when (and when NOT) to use it, a real framework that ships it, and trade-offs.

1 Overview --- What It Is

A **design pattern** is a named, reusable *template* for solving a recurring object-oriented design problem. It is **not** a library, a framework, or copy-pasteable code. It is a description of *which classes to introduce, how they collaborate, and which responsibilities to assign to which object* — captured at a level of abstraction you can re-apply across wildly different domains.

The canonical 23 patterns come from the 1994 book *Design Patterns: Elements of Reusable Object-Oriented Software* by Gamma, Helm, Johnson, and Vlissides — the **Gang of Four (GoF)**. They sorted patterns into three families by *what aspect of the system they manage*:

CREATIONAL "how objects are created & who owns creation"	STRUCTURAL "how objects are composed into larger structures"	BEHAVIORAL "how objects communicate & distribute work"
Singleton	Adapter	Strategy
Factory Method	Decorator	Observer
Abstract Factory	Facade	Command
Builder	Proxy	State
Prototype	Composite	Template Method
	Bridge	Iterator
	Flyweight	Chain of Responsibility
		Mediator
		Memento
		Visitor
		Interpreter

Each pattern is described by four things:

Element	Meaning	Example (Strategy)
Intent	The one-line purpose	"Make a family of algorithms interchangeable"
Problem / Forces	The tension it resolves	"Algorithm must vary at runtime without editing the caller"
Structure	The participating classes + relationships	Context holds a Strategy interface, swaps concrete impls
Consequences	The trade-offs you accept	More classes, runtime flexibility, OCP compliance

Beyond GoF, **modern/architectural patterns** — Dependency Injection, MVC/MVP/MVVM, Repository, Null Object, Object Pool — appear constantly in real codebases and interviews, so they get full treatment here too.

Interview takeaway: A pattern is *a name for a shape that good OO code naturally takes*. Your job in an interview is to **recognize the shape** the requirement is pushing you toward, name it, and justify the trade-off — not to recite UML.

2 Why It Exists

Before patterns, every experienced designer kept re-deriving the *same* solutions to the *same* structural problems — and every team described those solutions with different words. Patterns exist to fix three things:

1. **Re-invention.** The problem "I need to add behavior to objects at runtime without a combinatorial explosion of subclasses" has been solved thousands of times. The Decorator pattern is the distilled, vetted answer. You shouldn't re-discover it under interview pressure.
2. **Communication.** Saying "*wrap it in a Strategy*" transmits an entire micro-architecture — an interface, a context that delegates, concrete implementations, runtime swapping — in three words. Patterns are a **compression codec for design intent**. On a team, this is the difference between a 5-minute alignment and a 50-minute argument.
3. **Principle-in-practice.** Every pattern is SOLID + composition-over-inheritance applied to a *specific shape* of problem. Strategy is the Open/Closed Principle made concrete. Observer is loose coupling between producer and consumer. Patterns are how abstract principles become muscle memory.

Symptoms a codebase is *missing* the right patterns:

- A switch (type) statement that grows every time a new type is added (**needs Strategy/Factory/State**).
- Adding a feature requires editing 12 files (**shotgun surgery — needs better encapsulation of what varies**).
- A class with 4,000 lines doing everything (**God object — needs SRP + delegation**).
- Subclass explosion: BufferedEncryptedCompressedStream (**needs Decorator**).

Patterns are *discovered*, not *invented*. They are the recurring crystal shapes good object-oriented code condenses into under the pressure of change. The skill is recognizing the shape early.

3 Why FAANG Cares

Design patterns sit at the intersection of three signals interviewers explicitly grade: **code quality**, **extensibility**, and **judgment**.

- **Amazon** — Bar Raisers tie clean OOD to "Insist on Highest Standards" and "Dive Deep." In machine-coding / LLD rounds (parking lot, ride-share, in-memory cache) they watch whether you reach for Strategy/Factory/Observer *appropriately* and whether you can extend your own design mid-interview without rewriting it (OCP in action).
- **Google** — Grades *code quality* as a distinct hiring signal. The bar is **right-sized abstraction**: do you know when a pattern earns its indirection vs. when a plain function is better? Over-patterning is penalized as hard as under-designing.
- **Meta** — "Move fast" without leaving landmines. LLD rounds test whether your design absorbs new requirements gracefully. Patterns are the mechanism; the judgment to *not* use them is the senior signal.
- **Microsoft** — Heavy on SOLID and extensibility. Expect "now add requirement X" to your design — patterns are what make that a 2-line answer instead of a refactor.
- **Apple** — Cares about ownership semantics, interface design, and correct composition vs. inheritance choices.

The specific signals being tested:

1. Can you **map a requirement to a pattern**? ("Pricing must be pluggable" → Strategy.)
2. Can you **articulate the trade-off**? ("Decorator avoids subclass explosion but produces many small wrapper objects.")
3. Do you know **when NOT to use a pattern**? (Two states that never change → if/else beats State.)
4. Can you **read framework code**? (Spring beans = Factory + Singleton + Proxy; React = Observer + Composite.)
5. Do you recognize **anti-patterns** and know the refactor that fixes them?

Interview signal: Naming a pattern *and* justifying its trade-off marks a senior engineer. Naming a pattern you don't need marks a junior one.

4 Core Concepts

Patterns are not random tricks — they are all derived from a small set of **design principles**. Master these five and most patterns become obvious consequences.

4.1 1. Program to an Interface, Not an Implementation

The foundational GoF mantra. Depend on **abstractions** (interfaces / abstract base classes), never on concrete classes. This is the **Dependency Inversion Principle (DIP)** in one sentence.

```
1 // BAD — caller welded to a concrete class; swapping it edits this code
2 ArrayList<Order> orders = new ArrayList<>();
3
4 // GOOD — caller depends only on the List contract; impl is swappable
5 List<Order> orders = new ArrayList<>(); // could be LinkedList, CopyOnWriteArrayList...
```

The payoff: you can substitute mocks in tests, swap a `MySQLRepository` for an `InMemoryRepository`, and the calling code never changes. **Every creational pattern exists to help you obey this rule** — they let the caller request an abstraction and receive a concrete object without naming the concrete class.

4.2 2. Favor Composition Over Inheritance

Inheritance ("white-box reuse") exposes the parent's internals to the child, creating the **fragile base class problem**: a change to the parent silently breaks children. Composition ("black-box reuse") delegates to a held object through its public interface — looser, runtime-swappable.

```
1 # Inheritance — Logger is permanently a FileHandler; can't switch to network logging
2 class Logger(FileHandler): ...
3
4 # Composition — swap the handler at runtime; Logger is decoupled from storage
5 class Logger:
6     def __init__(self, handler: LogHandler): # FileHandler | ConsoleHandler | KafkaHandler
7         self._handler = handler
8     def log(self, msg): self._handler.emit(msg)
```

Most structural and behavioral patterns (Strategy, Decorator, Bridge, State, Adapter, Proxy) are **composition** mechanisms. When you find yourself subclassing purely to reuse code, stop

— compose instead.

4.3 3. Encapsulate What Varies

Identify the part of the system that **changes** (the algorithm, the product type, the notification target) and isolate it behind a stable interface. The stable rest of the system then *never changes* when the varying part does — this **is** the Open/Closed Principle (open for extension, closed for modification).

The whole catalog of patterns, in one sentence:
 "Find what varies, wrap it behind an interface, and depend on the interface."
 - Algorithm varies → Strategy
 - Object created varies → Factory
 - State-dependent behavior varies → State
 - Wrapped responsibilities vary → Decorator

4.4 4. SOLID Recap (the principles patterns enforce)

Letter	Principle	One-liner	Pattern that embodies it
S	Single Responsibility	One class, one reason to change	Command (one action per object)
O	Open/Closed	Extend by adding code, not editing	Strategy, Factory, Decorator
L	Liskov Substitution	Subtypes honor the base contract	Template Method, Null Object
I	Interface Segregation	Many small interfaces > one fat one	Adapter, Visitor
D	Dependency Inversion	Depend on abstractions	Abstract Factory, DI

Mnemonic: A pattern that *violates* SOLID is being misused. If your "Singleton" has 20 unrelated methods, it's also a God object (SRP violation) — the pattern label doesn't excuse it.

4.5 5. The Hollywood Principle --- "Don't call us, we'll call you"

Inversion of Control (IoC): high-level code defines the *skeleton* and calls down into low-level hooks; low-level components do not call up into the framework. This is the heart of **Template Method** (base class calls your overridden steps), **Observer** (subject calls your `update`), and **Dependency Injection** (the container constructs and hands you your dependencies).

Traditional flow: your code → library
 Hollywood / IoC: framework → your hooks (framework owns the control loop)

This single inversion is *why* frameworks (Spring, JUnit, React) feel different from libraries: you fill in blanks, the framework runs the show.

Interview takeaway: If asked "what's the principle behind this pattern?", almost every correct answer is one of: *encapsulate what varies (OCP)*, *program to interfaces (DIP)*, *favor composition*, or *invert control (Hollywood)*.

5 Creational Patterns

Creational patterns abstract the **instantiation process** — they let a system be independent of *how its objects are created, composed, and represented*. The common thread: the caller asks for an abstraction and receives a concrete object **without writing** `new ConcreteClass()` itself.

5.1 Singleton

Intent: Ensure a class has exactly one instance and provide a single global access point to it.

Problem it solves: Some resources must be unique and shared: a connection pool, an in-memory cache, a logger, an app-wide config registry. Creating two would be wrong (two pools double the DB connections) or wasteful. You need *one* instance and a way for any code to reach it.

Structure:

```
+-----+
| Singleton |
+-----+
| - instance: Singleton | (static, private)
| - Singleton() | (private ctor - no outside `new`)
+-----+
| + getInstance(): S | (static accessor; lazily/eagerly creates)
+-----+
      ▲ returns the same object every call
```

The four canonical thread-safe Java variants (this is the #1 Singleton interview question):

```
1 // 1) EAGER - instance built at class-load. Simple, thread-safe by JVM guarantee.
2 //   Downside: built even if never used.
3 public class EagerConfig {
4     private static final EagerConfig INSTANCE = new EagerConfig();
5     private EagerConfig() {}
6     public static EagerConfig get() { return INSTANCE; }
7 }
8
9 // 2) DOUBLE-CHECKED LOCKING - lazy + thread-safe. `volatile` is MANDATORY
10 //   (without it, partially-constructed object can leak due to reordering).
11 public class DclConfig {
12     private static volatile DclConfig instance; // volatile = no reordering
13     private DclConfig() {}
14     public static DclConfig get() {
15         if (instance == null) { // 1st check (no lock, fast path)
16             synchronized (DclConfig.class) {
17                 if (instance == null) { // 2nd check (locked)
18                     instance = new DclConfig();
19                 }
20             }
21         }
22         return instance;
23     }
24 }
25
26 // 3) BILL PUGH / INITIALIZATION-ON-DEMAND HOLDER - best of both.
27 //   Lazy (holder class loads only on first get()), thread-safe (JVM class-init lock),
28 //   no synchronization cost on the hot path. The recommended idiom.
29 public class HolderConfig {
30     private HolderConfig() {}
31     private static class Holder { static final HolderConfig I = new HolderConfig(); }
```



```

32     public static HolderConfig get() { return Holder.I; }
33 }
34
35 // 4) ENUM – Joshua Bloch's preferred form. Thread-safe, serialization-safe,
36 //     reflection-proof (you cannot reflectively instantiate a second enum constant).
37 public enum EnumConfig {
38     INSTANCE;
39     public void load() { /* ... */ }
40 }

```

```

1 # Python idiom – __new__ guard (GIL makes the simple version usually fine, but
2 # add a lock for correctness under true parallelism / asyncio)
3 import threading
4 class Config:
5     _instance = None
6     _lock = threading.Lock()
7     def __new__(cls):
8         if cls._instance is None:
9             with cls._lock:
10                 if cls._instance is None:
11                     cls._instance = super().__new__(cls)
12         return cls._instance

```

When to use: Genuinely-single shared resources — connection/thread pools, caches, config, logging facades. **When NOT to use:** As a convenient global variable. Singletons are *hidden* dependencies (a method that calls `Config.get()` doesn't declare it needs config), they poison unit tests (can't inject a mock; state leaks between tests), and they couple everything to a concrete class. **Prefer dependency injection of a single instance** managed by a container — you keep "one instance" without the global-access pathologies.

Real framework example: `java.lang.Runtime.getRuntime()`, `java.awt.Desktop.getDesktop()`, Spring beans (singleton-scoped by default, but DI-managed so they're *testable* singletons).

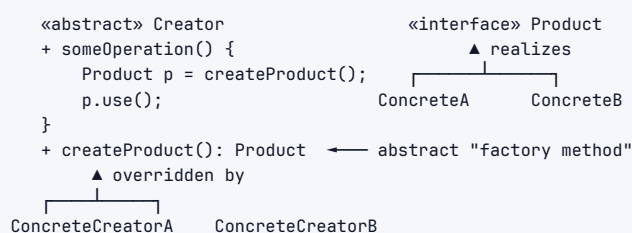
Trade-offs: Controlled single instance, lazy init possible, global reach. Hidden global state, hard to test, thread-safety complexity, violates SRP if it accretes logic, can mask poor design ("everything is a singleton").

5.2 Factory Method

Intent: Define an interface for creating an object, but let **subclasses** decide which concrete class to instantiate. Deferring instantiation to subclasses.

Problem it solves: A `LogisticsApp` defines the workflow `planDelivery()` which needs a `Transport`. But the *type* of transport depends on the subclass: `RoadLogistics` creates `Truck`, `SeaLogistics` creates `Ship`. The base class can't hard-code `new Truck()` — it would break sea logistics. So it calls an abstract `createTransport()` hook that subclasses override.

Structure:



```
createProduct()=A    createProduct()=B
```

```

1 interface Transport { void deliver(); }
2 class Truck implements Transport { public void deliver(){ System.out.println("by road"); } }
3 class Ship implements Transport { public void deliver(){ System.out.println("by sea"); } }
4
5 abstract class Logistics {
6     public void planDelivery() {           // the fixed workflow...
7         Transport t = createTransport();    // ...uses the factory hook
8         t.deliver();
9     }
10    protected abstract Transport createTransport(); // subclasses decide the type
11 }
12 class RoadLogistics extends Logistics {
13     protected Transport createTransport() { return new Truck(); }
14 }
15 class SeaLogistics extends Logistics {
16     protected Transport createTransport() { return new Ship(); }
17 }

```

When to use: When a class can't anticipate the concrete type it must create, or you want subclasses to specify it. When you want to decouple object construction from its use (DIP/OCP).
When NOT to use: When there's only ever one concrete product — a direct constructor is clearer (YAGNI).

Real framework example: `java.util.Calendar.getInstance()`, `java.text.NumberFormat.getInstance()`, the JDBC `Connection.createStatement()`, Django's `Model.objects.create()`.

Trade-offs: Decouples client from concrete classes, single place to change creation, OCP-friendly. Requires a subclass per product variation, can proliferate classes for trivial creation.

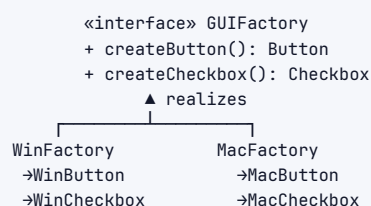
Factory Method vs. simple/static factory: The GoF Factory Method uses *inheritance* (subclass overrides the creator). A "simple factory" — one `create(type)` method with a switch — is not a GoF pattern but is common and fine for small cases. Mention the distinction in interviews.

5.3 Abstract Factory

Intent: Provide an interface for creating **families of related objects** without specifying their concrete classes — and guarantee the products are compatible with each other.

Problem it solves: A cross-platform UI must render a `Button` *and* a `Checkbox`. On Windows both must be Windows-style; on macOS both must be Mac-style. You must never mix a `WinButton` with a `MacCheckbox`. An Abstract Factory bundles the creation of a *whole family* so a single factory choice guarantees a consistent set.

Structure:



Client depends ONLY on GUIFactory + Button + Checkbox interfaces.
 Pick the factory once → every product matches.

```

1  from abc import ABC, abstractmethod
2
3  class Button(ABC):
4      @abstractmethod
5      def render(self): ...
6  class Checkbox(ABC):
7      @abstractmethod
8      def render(self): ...
9
10 class WinButton(Button):
11     def render(self): print("[Win Button]")
12 class WinCheckbox(Checkbox):
13     def render(self): print("[Win Checkbox]")
14 class MacButton(Button):
15     def render(self): print("(Mac Button)")
16 class MacCheckbox(Checkbox):
17     def render(self): print("(Mac Checkbox)")
18
19 class GUIFactory(ABC):
20     @abstractmethod
21     def create_button(self) → Button: ...
22     @abstractmethod
23     def create_checkbox(self) → Checkbox: ...
24
25 class WinFactory(GUIFactory):
26     def create_button(self): return WinButton()
27     def create_checkbox(self): return WinCheckbox()
28 class MacFactory(GUIFactory):
29     def create_button(self): return MacButton()
30     def create_checkbox(self): return MacCheckbox()
31
32 def build_ui(factory: GUIFactory):      # client is OS-agnostic
33     factory.create_button().render()
34     factory.create_checkbox().render()

```

When to use: Multiple product families that must stay internally consistent (UI toolkits per OS/theme, DB driver suites per vendor, cloud-provider SDKs). **When NOT to use:** Only one family exists today (YAGNI), or products aren't actually related — then a Factory Method per product is simpler.

Real framework example: `javax.xml.parsers.DocumentBuilderFactory`, `javax.xml.transform.TransformerFactory`, AWT's Toolkit, cross-platform UI kits (Qt, Swing L&F).

Trade-offs: Guarantees compatible product families, isolates concrete classes, swap a whole family by swapping one factory. Adding a *new product type* (e.g., a slider) forces editing every factory interface + impl — the pattern is rigid against new product *kinds* while flexible against new *families*.

Factory Method vs. Abstract Factory: Factory Method creates **one** product via inheritance; Abstract Factory creates a **family** of related products via composition (a factory object with several create-methods). Abstract Factory is frequently *implemented using* Factory Methods.

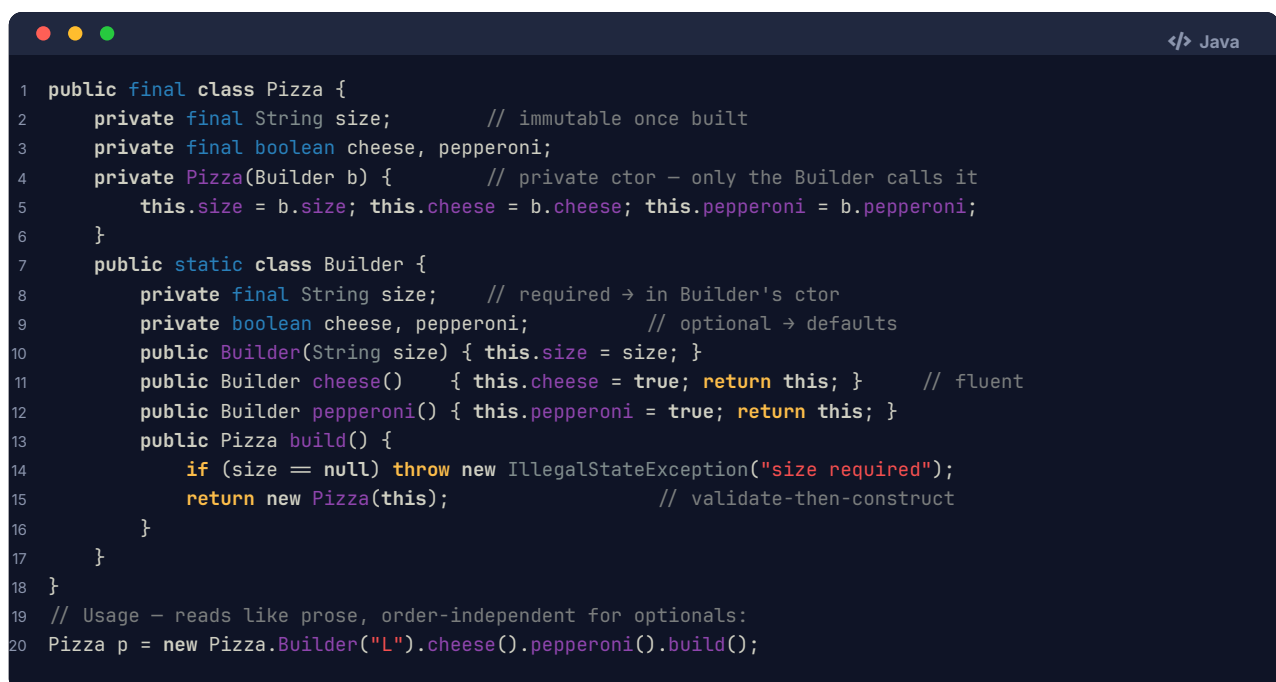
5.4 Builder

Intent: Separate the construction of a complex object from its representation, so the same construction process can produce different representations — and so an object with many parameters can be built step-by-step.

Problem it solves: Telescoping constructors. An object with 8 fields (3 required, 5 optional) forces either a dozen overloaded constructors (`Pizza(size)`, `Pizza(size, crust)`, `Pizza(size, crust, cheese)`...) or a single constructor with 8 positional args you can't read (`new Pizza("L", "thin", true, false, true, null, 2, "extra")` — which boolean is which?). Builder gives a fluent, self-documenting, validated construction path and supports **immutable** results.

Structure:

```
Client → Builder.size("L").crust("thin").cheese().build() → Pizza (immutable)
      ↳ each step returns 'this' (fluent chaining)
      ↳ build() validates invariants, then constructs
```



```
1 public final class Pizza {
2     private final String size;          // immutable once built
3     private final boolean cheese, pepperoni;
4     private Pizza(Builder b) {          // private ctor — only the Builder calls it
5         this.size = b.size; this.cheese = b.cheese; this.pepperoni = b.pepperoni;
6     }
7     public static class Builder {
8         private final String size;      // required → in Builder's ctor
9         private boolean cheese, pepperoni; // optional → defaults
10        public Builder(String size) { this.size = size; }
11        public Builder cheese() { this.cheese = true; return this; } // fluent
12        public Builder pepperoni() { this.pepperoni = true; return this; }
13        public Pizza build() {
14            if (size == null) throw new IllegalStateException("size required");
15            return new Pizza(this);        // validate-then-construct
16        }
17    }
18 }
19 // Usage — reads like prose, order-independent for optionals:
20 Pizza p = new Pizza.Builder("L").cheese().pepperoni().build();
```

When to use: Many optional/configurable parameters; you want immutability; construction needs validation or multiple steps; you want to build *different representations* with the same steps (a Director orchestrating builders). **When NOT to use:** Objects with 2–3 fields — a plain constructor or a small record is simpler.

Real framework example: `StringBuilder`, `java.net.http.HttpRequest.Builder`, `Stream.Builder`, Lombok `@Builder`, Guava's `ImmutableList.builder()`, Kotlin's named/default args (a language feature that *replaces* Builder), protobuf message builders.

Trade-offs: Readable construction, immutability, validation in one place, no telescoping ctors. More boilerplate (a whole Builder class), slight object-allocation overhead, overkill for simple objects.

5.5 Prototype

Intent: Specify the kinds of objects to create using a prototypical instance, and create new objects by **copying (cloning)** this prototype rather than constructing from scratch.

Problem it solves: Object creation is expensive (a DB round-trip, heavy computation, deserialization) or the exact class is decided at runtime. Instead of re-running the expensive setup,

you keep a pre-built prototype and `clone()` it, then tweak the copy. Also useful when a system must be independent of how its products are created.

Structure:

```
«interface» Prototype { clone(): Prototype }
    ▲
    ConcreteShape { clone(): returns a copy of itself }

registry: { "warrior": warriorPrototype, "mage": magePrototype }
spawn("warrior") → warriorPrototype.clone()    (cheap copy, no DB hit)
```



```
1 import copy
2
3 class GameEnemy:
4     def __init__(self, sprite, stats, ai_tree):
5         self.sprite = sprite          # imagine: loaded from disk (expensive)
6         self.stats = stats
7         self.ai_tree = ai_tree        # imagine: compiled behavior tree (expensive)
8
9     def clone(self) → "GameEnemy":
10        return copy.deepcopy(self)    # deep vs shallow matters! (see trade-offs)
11
12 # Build the costly prototype ONCE, then spawn a wave cheaply:
13 orc_prototype = GameEnemy(load_sprite("orc.png"), {"hp": 50}, compile_ai("orc.bt"))
14 wave = [orc_prototype.clone() for _ in range(100)]    # no disk/compile per enemy
15 wave[0].stats["hp"] = 80                          # tweak the copy independently
```

When to use: Costly construction you can amortize via copy; many similar objects differing in a few fields; the concrete type is determined at runtime (clone a registered prototype by key).
When NOT to use: Cheap-to-construct objects, or objects with complex reference graphs / external resources (file handles, sockets) where copying semantics are murky.

Real framework example: Java's `Cloneable` / `Object.clone()`, JavaScript's `Object.create(proto)` and prototypal inheritance, undo systems that snapshot via clone, ML pipeline `clone()` (scikit-learn `clone(estimator)` copies hyperparameters).

Trade-offs: Avoids expensive re-init, decouples from concrete class, runtime-configurable.
Shallow vs. deep copy is the classic bug — a shallow clone shares mutable sub-objects (two "copies" mutating the same list). Cloning objects with cycles or external resources is hard. `Cloneable` in Java is widely considered a flawed API.

5.6 Creational Patterns --- Comparison

Pattern	What it abstracts	Reach for it when
Singleton	<i>how many</i> instances (exactly one)	Logger, config, pool — sparingly; prefer DI
Factory Method	<i>which subclass</i> creates the product	One product, concrete type chosen by subclass
Abstract Factory	a <i>family</i> of compatible products	Cross-platform/pluggable suites that must match
Builder	<i>step-by-step</i> assembly of one complex object	Many optional fields, immutability, validation
Prototype	creation via <i>copy</i> of an exemplar	Expensive construction, runtime-decided type

6 Structural Patterns

Structural patterns are about **composing classes and objects into larger structures** while keeping those structures flexible and efficient. The recurring theme: a wrapper or tree that lets objects collaborate without rigid coupling.

6.1 Adapter

Intent: Convert the interface of a class into another interface clients expect. Lets classes with incompatible interfaces work together.

Problem it solves: Your code is written against a `PaymentGateway` interface with `pay(amount)`. You must integrate a third-party `StripeSdk` that exposes `makeCharge(cents, currency)`. You can't edit the SDK and you don't want to rewrite your call sites. An Adapter wraps the SDK and presents the `PaymentGateway` interface, translating each call.

Structure:

```
Client → «interface» Target.request()
      ▲ realizes
Adapter —holds—> Adaptee (incompatible 3rd-party class)
request() { adaptee.specificRequest(); } // translates the call
```

```
1 interface PaymentGateway { boolean pay(double dollars); } // what our code expects
2
3 class StripeSdk { // 3rd-party, can't change
4     boolean makeCharge(long cents, String currency) { /* ... */ return true; }
5 }
6
7 class StripeAdapter implements PaymentGateway { // the bridge
8     private final StripeSdk stripe;
9     StripeAdapter(StripeSdk s) { this.stripe = s; }
10    public boolean pay(double dollars) {
11        return stripe.makeCharge(Math.round(dollars * 100), "USD"); // translate
12    }
13 }
14 // Client code stays clean: PaymentGateway g = new StripeAdapter(new StripeSdk());
```

When to use: Integrating third-party libs, legacy code, or two subsystems with mismatched interfaces; making old code satisfy a new interface without rewriting it. **When NOT to use:** When you control both sides — just align the interfaces. When the impedance mismatch is so large the adapter would reimplement the adaptee (that's a rewrite, not an adapter).

Real framework example: `java.io.InputStreamReader` (adapts `byte InputStream` → `char Reader`), `java.util.Arrays.asList()` (`array` → `List`), `java.util.Collections.list(Enumeration)` (old `Enumeration` → modern `List`), `Slf4j` bridges (`slf4j-log4j12`).

Trade-offs: Reuse incompatible code, isolate the dependency in one place, SRP (translation logic localized). Extra indirection; deep adapter chains get confusing; can hide a mismatch that should be fixed at the source.

Object Adapter (composition, shown above) vs. Class Adapter (multiple inheritance).

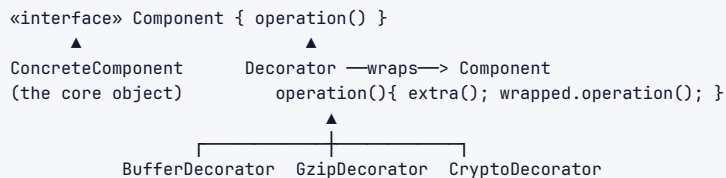
Object adapter is preferred and the only option in single-inheritance languages like Java for combining with concrete adaptees.

6.2 Decorator

Intent: Attach additional responsibilities to an object **dynamically**. A flexible alternative to subclassing for extending functionality.

Problem it solves: You have a `DataSource` and want optional, *combinable* features: compression, encryption, buffering. With inheritance you'd need a class per combination — `CompressedSource`, `EncryptedSource`, `CompressedEncryptedSource`, `BufferedCompressedEncryptedSource`... an exponential explosion. Decorator wraps the object in layers, each adding one responsibility, composed at runtime in any order.

Structure:



```

1 interface DataSource { void write(String data); String read(); }
2
3 class FileDataSource implements DataSource { // the core
4     public void write(String d){ /* to file */ }
5     public String read(){ return "raw"; }
6 }
7
8 abstract class DataSourceDecorator implements DataSource { // base wrapper
9     protected final DataSource wrappee;
10    DataSourceDecorator(DataSource s){ this.wrappee = s; }
11    public void write(String d){ wrappee.write(d); }
12    public String read(){ return wrappee.read(); }
13 }
14
15 class EncryptionDecorator extends DataSourceDecorator {
16     EncryptionDecorator(DataSource s){ super(s); }
17     public void write(String d){ super.write(encrypt(d)); } // add behavior, then delegate
18     public String read(){ return decrypt(super.read()); }
19     private String encrypt(String s){ return "enc(" + s + ")"; }
20     private String decrypt(String s){ return s; }
21 }
22 // Stack at runtime, any order:
23 DataSource src = new EncryptionDecorator(new CompressionDecorator(new FileDataSource()));
  
```

The canonical Java I/O Streams example (the textbook real-world Decorator):

```

1 // Each constructor wraps the previous, adding one capability:
2 BufferedReader r = new BufferedReader( // adds buffering
3     new InputStreamReader( // adapts bytes→chars (Adapter, actually)
4         new FileInputStream("data.txt"))); // the core byte source
5 // Compose freely: GZIPInputStream, CipherInputStream, DataInputStream ... all decorators.
  
```

When to use: Optional, stackable, order-flexible responsibilities; you want to add behavior without touching the original class or exploding subclasses (OCP). **When NOT to use:** When you need to inspect the *concrete* wrapped type at runtime (decorators hide it); when there's only one fixed feature (just subclass); when ordering bugs would be subtle.

Real framework example: Java I/O streams (the definitive example), `java.util.Collections.synchronized`

/ `unmodifiableList()`, Python's `@functools.lru_cache` and other function decorators, HTTP middleware chains (Express, Django middleware), `javax.servlet.http.HttpServletRequestWrapper`.

Trade-offs: Runtime composition, avoids subclass explosion, single-responsibility layers, OCP. Many small objects (debugging a 5-layer stack is painful), order can matter and be non-obvious, hard to remove a specific decorator from the middle of a stack, identity checks (`instanceof`) break through wrappers.

Decorator vs. Inheritance: inheritance fixes the augmentation at compile time and can't combine; Decorator composes at runtime and stacks arbitrarily. **Decorator vs. Proxy:** identical structure (both wrap, same interface) — *intent* differs: Decorator *adds responsibilities*; Proxy *controls access*.

6.3 Facade

Intent: Provide a unified, simplified interface to a set of interfaces in a subsystem. Makes the subsystem easier to use.

Problem it solves: Converting a video involves codecs, audio mixers, bitrate readers, format detectors — a dozen subsystem classes with intricate setup order. Most clients just want `convert(file, "mp4")`. A Facade exposes one simple method and orchestrates the messy internals, decoupling clients from the subsystem.

Structure:

```
Client → Facade.watchMovie()
      |
      | orchestrates:
      | - amp.on(); amp.setVolume(5)
      | - projector.on(); projector.wideScreen()
      | - player.on(); player.play(movie)
      |
      | (subsystem classes remain usable directly for power users)
```

```

1 class Amplifier:
2     def on(self): print("Amp on")
3     def set_volume(self, v): print(f"Volume {v}")
4 class Projector:
5     def on(self): print("Projector on")
6     def wide(self): print("Wide screen")
7 class Player:
8     def on(self): print("Player on")
9     def play(self, m): print(f"Playing {m}")
10
11 class HomeTheaterFacade:           # one simple front
12     def __init__(self, amp, proj, player):
13         self._amp, self._proj, self._player = amp, proj, player
14     def watch_movie(self, movie):   # hides ordering & coordination
15         self._amp.on(); self._amp.set_volume(5)
16         self._proj.on(); self._proj.wide()
17         self._player.on(); self._player.play(movie)
18
19 HomeTheaterFacade(Amplifier(), Projector(), Player()).watch_movie("Inception")

```

When to use: A complex subsystem with intricate APIs; you want a simple entry point and to reduce client↔subsystem coupling; layering an architecture (each layer exposes a facade to the next). **When NOT to use:** When the subsystem is already simple (the facade adds a useless layer), or when clients genuinely need fine-grained control (then expose both).

Real framework example: SLF4J (`LoggerFactory.getLogger()` fronts `log4j/logback`), `javax.faces.context.FacesContext`, Spring's `JdbcTemplate` (facade over raw JDBC), AWS SDK

service clients, a REST gateway fronting microservices.

Trade-offs: Simpler client code, decouples clients from subsystem internals, defines a clear layer boundary. Can become a God object if it accretes logic; risks hiding power users need; another layer to maintain.

Facade vs. Adapter: Adapter makes one interface *match* an expected one (interface conversion); Facade *simplifies* many interfaces into a new, easier one (interface reduction). Facade is about ease; Adapter is about compatibility.

6.4 Proxy

Intent: Provide a surrogate or placeholder for another object to **control access** to it — same interface as the real subject, with added control logic.

Problem it solves: You need to do *something around* access to an object without changing the object or its clients: defer its expensive creation, check permissions, cache results, log, or reach it across a network. A Proxy implements the same interface and inserts that logic.

Flavors:

Flavor	Purpose	Example
Virtual	Lazy-create an expensive object on first use	High-res image not loaded until displayed
Protection	Access control / authorization	Reject delete() unless caller is admin
Remote	Local stand-in for an object in another process	RMI/gRPC client stub
Caching	Memoize expensive results	Cache HTTP/DB responses
Smart reference	Extra bookkeeping (ref-count, logging)	Hibernate lazy proxies, AOP

Structure:

```

Client → «interface» Subject { request() }
      ▲
      RealSubject
      (heavy/remote)

      ▲
      Proxy —holds—> RealSubject
      request(){ check/lazy/cache; real.request(); }
  
```

```

1 from abc import ABC, abstractmethod
2
3 class Image(ABC):
4     @abstractmethod
5     def display(self): ...
6
7 class RealImage(Image):
8     def __init__(self, filename):
9         self.filename = filename
10        self._load_from_disk()      # EXPENSIVE — happens at construction
11    def _load_from_disk(self): print(f"Loading {self.filename} (heavy I/O)")
12    def display(self): print(f"Displaying {self.filename}")
13
14 class ImageProxy(Image):          # VIRTUAL proxy — defers the heavy load
15     def __init__(self, filename):
16         self.filename = filename
17         self._real = None
18     def display(self):
  
```

```

19     if self._real is None:           # create only on first actual use
20         self._real = RealImage(self.filename)
21         self._real.display()
22
23 img = ImageProxy("huge.png")        # no disk I/O yet
24 img.display()                       # NOW it loads, then displays
25 img.display()                       # already loaded – just displays

```

When to use: Lazy initialization of heavy objects, access control, remote access, caching, instrumentation — any “do work around access” need. **When NOT to use:** When the added latency/indirection isn't justified; when a Decorator (adds behavior) or a cache library would be clearer for the specific need.

Real framework example: Hibernate lazy-loading proxies, Spring AOP / @Transactional (dynamic proxy wraps the bean to open/commit transactions), java.lang.reflect.Proxy (dynamic proxies), gRPC/RMI stubs, mock objects in tests (a proxy that records calls).

Trade-offs: Control access transparently, lazy/remote/caching without changing clients, separation of concerns. Extra indirection/latency, response-time can become unpredictable (lazy load surprises), more classes; remote proxies hide network failures behind a local-looking call.

6.5 Composite

Intent: Compose objects into **tree structures** to represent part-whole hierarchies. Lets clients treat individual objects and compositions of objects **uniformly**.

Problem it solves: A file system has files (leaves) and directories (which contain files *and* directories). You want size() or print() to work the same whether you call it on a single file or a whole directory tree — without the client writing if (isDirectory) ... else ... everywhere. Composite gives leaves and containers a common interface and lets containers recurse into children.

Structure:

```

«interface» Component { operation() }
    ▲
Leaf
operation()
    ▲
Composite —contains 0..*—> Component
operation(){ for c in children: c.operation() }
add(c) / remove(c)

```

```

1  from abc import ABC, abstractmethod
2
3  class FsNode(ABC):
4      @abstractmethod
5      def size(self) → int: ...
6
7  class File(FsNode):           # LEAF
8      def __init__(self, name, bytes_): self.name, self._bytes = name, bytes_
9      def size(self): return self._bytes
10
11 class Directory(FsNode):       # COMPOSITE
12     def __init__(self, name): self.name, self._children = name, []
13     def add(self, node: FsNode): self._children.append(node)
14     def size(self):             # recurse uniformly – no type checks
15         return sum(child.size() for child in self._children)
16

```

```

17 root = Directory("root")
18 root.add(File("a.txt", 100))
19 docs = Directory("docs"); docs.add(File("b.pdf", 2000))
20 root.add(docs)
21 print(root.size()) # 2100 - works the same on file or directory

```

When to use: Genuine part-whole hierarchies (file systems, UI widget trees, org charts, arithmetic expression trees, graphics scene graphs). **When NOT to use:** Flat collections (no tree) or when leaves and composites are too different to share a meaningful interface (forcing `add()` onto a Leaf is awkward — a known design tension).

Real framework example: `java.awt.Container` / Swing JComponent trees, the DOM (nodes & elements), React component trees, `java.util.Map.Entry` collections, Composite UI in Android `ViewGroup/View`.

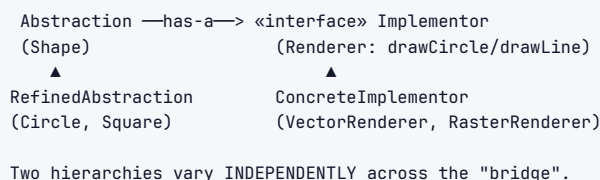
Trade-offs: Uniform treatment of leaf/composite, easy to add new node types, natural recursion. The shared interface can over-generalize (a `File` shouldn't really have `add()`); can make the design too permissive (type safety lost); deep trees risk stack overflow on naive recursion.

6.6 Bridge

Intent: Decouple an **abstraction** from its **implementation** so the two can vary independently — avoiding a Cartesian-product class explosion.

Problem it solves: You have Shape (Circle, Square) and rendering APIs (Vector, Raster). With inheritance you'd need `VectorCircle`, `RasterCircle`, `VectorSquare`, `RasterSquare` — and every new shape *or* new renderer multiplies the matrix. Bridge splits these into two hierarchies: Shape (abstraction) *has-a* Renderer (implementor). Now n shapes + m renderers = $n + m$ classes, not $n \times m$.

Structure:



```

1 interface Renderer { void renderCircle(float radius); } // implementor
2 class VectorRenderer implements Renderer {
3     public void renderCircle(float r){ System.out.println("vector circle r=" + r); }
4 }
5 class RasterRenderer implements Renderer {
6     public void renderCircle(float r){ System.out.println("pixels circle r=" + r); }
7 }
8
9 abstract class Shape { // abstraction
10     protected final Renderer renderer; // the BRIDGE (composition)
11     Shape(Renderer r){ this.renderer = r; }
12     abstract void draw();
13 }
14 class Circle extends Shape {
15     private final float radius;
16     Circle(Renderer r, float radius){ super(r); this.radius = radius; }

```

```

17     void draw(){ renderer.renderCircle(radius); }    // delegate across the bridge
18 }
19 // new shape OR new renderer added independently — no n×m explosion:
20 new Circle(new VectorRenderer(), 5).draw();
21 new Circle(new RasterRenderer(), 5).draw();

```

When to use: Two (or more) orthogonal dimensions that both vary (shape × renderer, message × transport, UI × theme, driver × platform); you want to avoid a combinatorial subclass matrix; you want to swap implementation at runtime. **When NOT to use:** Only one dimension varies (plain Strategy/inheritance suffices); premature when the second axis is hypothetical (YAGNI).

Real framework example: JDBC (the `Driver` abstraction bridges to vendor implementations), SLF4J (logging API bridged to log4j/logback backends), device-driver architectures, AWT peer architecture (`Button` ↔ platform-specific peer).

Trade-offs: Independent evolution of two hierarchies, no class explosion, runtime impl swap, hides impl from clients. Upfront complexity (two hierarchies + indirection); over-engineering if the second axis never materializes; can be confused with Strategy (it is structurally similar — intent: Bridge organizes a *structure*, Strategy swaps an *algorithm*).

6.7 Flyweight

Intent: Use sharing to support a **large number of fine-grained objects efficiently** — separate the **intrinsic** (shared, immutable) state from the **extrinsic** (context-specific) state, and reuse one shared object for all instances with the same intrinsic state.

Problem it solves: Rendering a document with a million characters, or a game with millions of trees/particles. Creating a distinct object per character (with font, glyph bitmap, kerning tables) blows up memory. Flyweight stores the *intrinsic* state (the glyph 'A', its font) **once** and shares it; the *extrinsic* state (position on the page) is passed in by the caller per use.

Structure:

```

FlyweightFactory: pool { "A@Arial": glyphA, "B@Arial": glyphB } ← shared, reused
getGlyph(char, font) → returns cached flyweight (creates once)

```

```

Glyph (flyweight): intrinsic = {char, font, bitmap} (immutable, shared)
render(x, y) ← extrinsic (x,y) PASSED IN, not stored

```

1,000,000 chars on screen → maybe 80 distinct Glyph objects in memory.

```

1 class Glyph:                                # FLYWEIGHT — intrinsic state only
2     def __init__(self, char, font):
3         self.char, self.font = char, font    # shared, immutable
4     def render(self, x, y):                  # extrinsic (x,y) passed per call
5         print(f"draw '{self.char}' ({self.font}) at ({x},{y})")
6
7 class GlyphFactory:                          # ensures sharing
8     _pool = {}
9     @classmethod
10    def get(cls, char, font):
11        key = (char, font)
12        if key not in cls._pool:              # create once, reuse forever
13            cls._pool[key] = Glyph(char, font)
14        return cls._pool[key]
15

```

```

16 # Rendering "AAB" — only TWO Glyph objects exist, reused:
17 for i, ch in enumerate("AAB"):
18     GlyphFactory.get(ch, "Arial").render(i * 10, 0)
19 print(len(GlyphFactory._pool))           # 2, not 3

```

When to use: Huge numbers of similar objects where most state is shareable and memory is the bottleneck (text glyphs, game particles/tiles, map markers, syntax-highlight tokens).
When NOT to use: Few objects; little shareable state; when the bookkeeping (factory + intrinsic/extrinsic split) costs more than the memory saved (premature optimization).

Real framework example: Java `Integer.valueOf()` cache (−128..127 share instances), String interning / the string pool, `Boolean.TRUE/FALSE`, Python's small-int caching, glyph caches in text-rendering engines.

Trade-offs: Massive memory savings for many fine-grained objects. Code complexity (intrinsic/extrinsic split is unintuitive), shared state **must be immutable / thread-safe** or you get cross-talk bugs, trades CPU (re-passing extrinsic state) for RAM.

6.8 Structural Patterns --- Comparison

Pattern	One-line intent	Wraps/structures	Key distinction
Adapter	Make incompatible interfaces fit	One object	Converts an interface
Decorator	Add responsibilities at runtime	One object (stackable)	<i>Adds</i> behavior, same interface
Proxy	Control access to an object	One object	<i>Controls</i> access, same interface
Facade	Simplify a complex subsystem	Many objects	<i>New, simpler</i> interface
Composite	Treat tree of objects uniformly	Tree of objects	Part-whole hierarchy
Bridge	Split abstraction from impl	Two hierarchies	Avoid $n \times m$ class explosion
Flyweight	Share to save memory	Pool of shared objects	Intrinsic vs extrinsic state

7 Behavioral Patterns

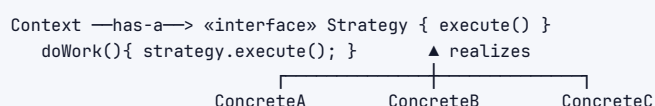
Behavioral patterns are about **algorithms and the assignment of responsibilities** between objects — how they communicate and distribute work. The theme: flexible communication that keeps senders and receivers loosely coupled.

7.1 Strategy

Intent: Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from the clients that use it.

Problem it solves: A route planner must compute routes by car, by walking, by public transit. Hard-coding `if (mode == CAR) ... elif (mode == WALK) ...` means every new mode edits the planner, and the planner balloons. Strategy extracts each algorithm into its own class behind a common interface; the planner holds a reference and delegates — adding a mode is a *new class, zero edits* (OCP).

Structure:



execute() execute() execute()

```

1 interface RouteStrategy { int estimate(String from, String to); } // algorithm
2
3 class CarStrategy implements RouteStrategy { public int estimate(String a,String b){ return 30; } }
4 class WalkStrategy implements RouteStrategy { public int estimate(String a,String b){ return 90; } }
5 class TransitStrategy implements RouteStrategy{ public int estimate(String a,String b){ return 45; } }
6
7 class Navigator { // CONTEXT
8     private RouteStrategy strategy;
9     Navigator(RouteStrategy s){ this.strategy = s; }
10    void setStrategy(RouteStrategy s){ this.strategy = s; } // swap at runtime
11    int route(String from, String to){ return strategy.estimate(from, to); }
12 }
13 Navigator nav = new Navigator(new CarStrategy());
14 nav.route("A", "B"); // 30
15 nav.setStrategy(new TransitStrategy()); // swapped - Navigator code untouched

```

When to use: Multiple interchangeable algorithms (pricing, compression codecs, payment methods, sorting orders, retry policies); you want to swap them at runtime; to eliminate a growing if/switch over algorithm type. **When NOT to use:** The algorithm never varies; in functional/dynamic languages a plain first-class function often is the strategy (don't ceremony-wrap it in a class).

Real framework example: `java.util.Comparator` (passed to `Collections.sort()`), `java.util.concurrent.RejectedExecutionHandler`, Spring Resource loaders, Python's key= callables, payment-gateway selection.

Trade-offs: OCP (new algorithm = new class), runtime swapping, testable in isolation, replaces conditionals. More classes/objects; clients must know the strategies exist to choose one; overkill when a lambda suffices.

Strategy is the poster child for composition + OCP + DIP. If an interviewer asks "name a pattern embodying Open/Closed," answer Strategy.

7.2 Observer

Intent: Define a one-to-many dependency so that when one object (the **subject**) changes state, all its dependents (**observers**) are notified and updated automatically.

Problem it solves: A stock price changes; a chart, an alert, and a logger must all react. Hard-wiring the stock object to call each consumer couples them tightly and means adding a consumer edits the stock. Observer lets consumers *subscribe*; the subject broadcasts to whoever is registered, knowing nothing about who they are.

Push vs. Pull:

Model	Subject sends...	Trade-off
Push	The changed data in the notification (<code>update(newPrice)</code>)	Simple for observers; subject must guess what they need
Pull	Just "something changed"; observer queries the subject (<code>update(subject) → subject.getPrice()</code>)	Observers fetch exactly what they need; more calls, looser data coupling

Structure:

```

Subject { subscribe(o) / unsubscribe(o) / notify() → for o in observers: o.update() }
    | holds list of
    ▼
«interface» Observer { update(...) }
    ▲
ConcreteObserverA  ConcreteObserverB  ...

```

```

1 from abc import ABC, abstractmethod
2
3 class Observer(ABC):
4     @abstractmethod
5     def update(self, price): ...           # PUSH model: data sent in
6
7 class Stock:                               # SUBJECT
8     def __init__(self):
9         self._observers = []
10        self._price = 0
11    def subscribe(self, o): self._observers.append(o)
12    def unsubscribe(self, o): self._observers.remove(o) # MUST exist (see leak below)
13    def set_price(self, p):
14        self._price = p
15        for o in self._observers:           # broadcast
16            o.update(p)
17
18 class PriceAlert(Observer):
19     def update(self, price): print(f"ALERT: {price}")
20 class Chart(Observer):
21     def update(self, price): print(f"chart += {price}")
22
23 s = Stock()
24 s.subscribe(PriceAlert()); s.subscribe(Chart())
25 s.set_price(150) # both notified automatically

```

The lapsed-listener leak (must mention in interviews): the subject holds a strong reference to every observer. If an observer is discarded without calling `unsubscribe()`, the subject keeps it alive forever — a **memory leak**, and it keeps receiving (now-useless) notifications. Fixes: explicit unsubscribe (try/finally), weak references (`WeakHashMap`/`WeakReference`), or scoping subscriptions to a lifecycle.

When to use: Event systems, MVC (model notifies views), reactive UIs, pub/sub, real-time dashboards, anywhere "many things must react to one thing changing." **When NOT to use:** When notification order matters (observers fire in undefined order); when update cascades could storm (debounce/batch); when a simple direct call to one consumer is enough.

Real framework example: Java `java.util.Observer` (deprecated — for a reason), Swing/AWT `addActionListener`, `PropertyChangeListener`, RxJava `Observable`, React state/useState re-render, Redux store subscriptions, Django signals, Kafka consumers (distributed Observer).

Trade-offs: Loose coupling producer↔consumer, dynamic subscription, broadcast for free, OCP. Lapsed-listener leaks, undefined notification order, hard-to-trace control flow ("who fired this update?"), update storms / cascade loops, debugging "spooky action at a distance."

7.3 Command

Intent: Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support **undo/redo**.

Problem it solves: A text editor's buttons, menu items, and keyboard shortcuts all trigger "actions." Wiring each UI element directly to editor methods couples them and makes undo

impossible. Command turns each action into an object with `execute()` and `undo()`. Now you can queue them, log them, replay them (macros), and maintain an undo/redo history stack.

Structure:

```
Invoker (button) —holds—> «interface» Command { execute(); undo() }
      ▲
      ConcreteCommand —calls—> Receiver (the editor)
      execute(){ receiver.action(); save state for undo }
History: stack<Command> → undo() pops & calls cmd.undo()
```

```
1 from abc import ABC, abstractmethod
2
3 class Command(ABC):
4     @abstractmethod
5     def execute(self): ...
6     @abstractmethod
7     def undo(self): ...
8
9 class Document:                                # RECEIVER
10     def __init__(self): self.text = ""
11     def insert(self, s): self.text += s
12     def delete(self, n): self.text = self.text[:-n]
13
14 class InsertCommand(Command):                  # CONCRETE COMMAND
15     def __init__(self, doc, s): self.doc, self.s = doc, s
16     def execute(self): self.doc.insert(self.s)
17     def undo(self): self.doc.delete(len(self.s)) # inverse op
18
19 class History:                                # INVOKER + undo/redo stacks
20     def __init__(self): self._done, self._undone = [], []
21     def run(self, cmd):
22         cmd.execute(); self._done.append(cmd); self._undone.clear()
23     def undo(self):
24         if self._done:
25             cmd = self._done.pop(); cmd.undo(); self._undone.append(cmd)
26     def redo(self):
27         if self._undone:
28             cmd = self._undone.pop(); cmd.execute(); self._done.append(cmd)
29
30 doc = Document(); h = History()
31 h.run(InsertCommand(doc, "hello")) # text="hello"
32 h.run(InsertCommand(doc, " world")) # text="hello world"
33 h.undo()                           # text="hello"
34 h.redo()                           # text="hello world"
```

When to use: Undo/redo, transactional operations, job/task queues (serialize a command and run it later/elsewhere), macro recording, GUI actions decoupled from handlers, request logging/replay. **When NOT to use:** Simple one-shot operations with no undo/queue/log requirement — a direct method call or a lambda is enough.

Real framework example: `java.lang.Runnable / Callable` (a command for a thread pool), `Swing Action`, `javax.swing.undo.UndoableEdit`, job queues (Celery tasks, Sidekiq jobs), the Command pattern underlies CQRS commands and event sourcing.

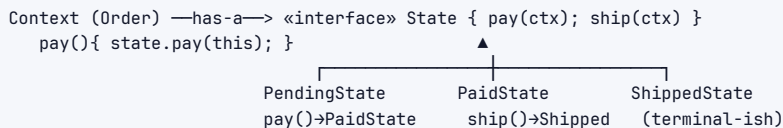
Trade-offs: Decouples invoker from receiver, enables undo/redo/queue/log/macro, SRP (one command = one action), composability (macro = list of commands). A class per action (proliferation), undo logic can be tricky (must capture inverse state — Memento often pairs here), memory for long histories.

7.4 State

Intent: Allow an object to alter its behavior when its internal state changes — the object appears to change its class.

Problem it solves: An order moves through PENDING → PAID → SHIPPED → DELIVERED, and each method (`pay()`, `ship()`, `cancel()`) behaves differently per state (`ship()` is illegal before payment). Encoding this as `if (status == PENDING) ... elif (status == PAID) ...` in *every* method produces parallel sprawling conditionals that all must be edited together. State makes each status a class that implements the behavior *and* decides the next transition.

Structure:



```

1 from abc import ABC, abstractmethod
2
3 class OrderState(ABC):
4     @abstractmethod
5     def pay(self, order): ...
6     @abstractmethod
7     def ship(self, order): ...
8
9 class Pending(OrderState):
10    def pay(self, order):
11        print("payment ok"); order.state = Paid()    # STATE decides the transition
12    def ship(self, order): print("can't ship - unpaid")
13
14 class Paid(OrderState):
15    def pay(self, order): print("already paid")
16    def ship(self, order):
17        print("shipping"); order.state = Shipped()
18
19 class Shipped(OrderState):
20    def pay(self, order): print("already paid")
21    def ship(self, order): print("already shipped")
22
23 class Order:                                # CONTEXT
24    def __init__(self): self.state = Pending()
25    def pay(self): self.state.pay(self)        # delegate to current state
26    def ship(self): self.state.ship(self)
27
28 o = Order()
29 o.ship() # "can't ship - unpaid"
30 o.pay()  # "payment ok"      (now Paid)
31 o.ship() # "shipping"       (now Shipped)
  
```

State vs. Strategy (the classic interview pair): structurally identical (a context delegates to a swappable helper interface). The **intent** differs:

	Strategy	State
Who picks the impl?	The client chooses an algorithm	The object transitions itself between states
Do the impls know each other?	No — independent algorithms	Yes — a state knows its successor states

	Strategy	State
How often does it change?	Usually set once, rarely mid-flow	Changes repeatedly as the object's lifecycle advances
Mental model	"How do I do X?"	"What can I do <i>now</i> ?"

When to use: Lifecycle-heavy objects (orders, tickets, connections, documents, vending machines, TCP connections, media players); many state-dependent methods; replacing a multi-method `switch(status)`. **When NOT to use:** 2–3 states that rarely change and have trivial behavior — an enum + if/else is clearer (don't over-engineer).

Real framework example: TCP connection states, `java.lang.Thread` states, workflow engines, game character state machines, parser states, Spring State Machine.

Trade-offs: Eliminates sprawling conditionals, each state's behavior + transitions localized (SRP/OCF), explicit legal transitions. Class per state (proliferation for many states), state-transition logic can be scattered across states (consider a transition table), overkill for trivial state sets.

7.5 Template Method

Intent: Define the **skeleton** of an algorithm in a base method, deferring some steps to subclasses. Subclasses redefine steps without changing the algorithm's structure.

Problem it solves: Exporting data to CSV vs. JSON shares a structure — fetch data, format it, write to file, notify — but the *format* and *write* steps differ. Duplicating the whole flow in each exporter violates DRY and risks the flows diverging. Template Method puts the invariant flow in a base method and calls overridable hooks for the varying steps (Hollywood Principle — the base calls down into your steps).

Structure:

```
«abstract» AbstractClass
  templateMethod() {
    step1();
    step2();
    hook();
  }
  }
  ▲ overrides step2()
  ConcreteA ConcreteB
```

← FINAL skeleton, defines the order
 ← concrete (shared)
 ← abstract (subclass fills in)
 ← optional override (default no-op)

```
</> Java
1 abstract class DataExporter {
2     public final void export(String file) { // TEMPLATE METHOD — final = fixed flow
3         var data = fetch(); // shared step
4         var out = format(data); // varies → abstract
5         write(out, file); // varies → abstract
6         onComplete(); // hook → optional override
7     }
8     protected List<String> fetch() { return List.of("a", "b"); } // shared
9     protected abstract String format(List<String> data); // subclass step
10    protected abstract void write(String content, String file); // subclass step
11    protected void onComplete() {} // hook (default no-op)
12 }
13 class CsvExporter extends DataExporter {
14     protected String format(List<String> d){ return String.join(",", d); }
15     protected void write(String c, String f){ /* write CSV */ }
16 }
17 class JsonExporter extends DataExporter {
18     protected String format(List<String> d){ return d.toString(); }
```

```

19     protected void write(String c, String f){ /* write JSON */ }
20     protected void onComplete(){ System.out.println("json done"); } // overrides hook
21 }

```

Template Method vs. Strategy: Template Method varies steps via **inheritance** (compile-time, subclass overrides) and keeps the algorithm skeleton in the base class; Strategy varies the **whole algorithm** via **composition** (runtime swap). Template Method = "fixed flow, pluggable steps"; Strategy = "pluggable whole."

When to use: Several variants share an algorithm skeleton with a few differing steps; you want to enforce the invariant ordering while allowing customization (frameworks do this heavily). **When NOT to use:** When composition/Strategy gives more flexibility (runtime swap, no inheritance coupling); when the "skeleton" is trivial.

Real framework example: `java.util.AbstractList/AbstractMap` (you implement `get/size`, get the rest free), `javax.servlet.http.HttpServlet.service()` (dispatches to your `doGet/doPost`), JUnit `setUp()/test/tearDown()`, Spring's `JdbcTemplate.execute()`, Android Activity lifecycle callbacks.

Trade-offs: DRY (shared flow once), enforces invariant structure, hooks for optional customization, framework-friendly. Inheritance coupling (fragile base class), inverted control can be hard to follow, hard to change the skeleton later, only one inheritance slot (Java single inheritance).

7.6 Iterator

Intent: Provide a way to access the elements of an aggregate object **sequentially** without exposing its underlying representation.

Problem it solves: A collection might be backed by an array, a linked list, a tree, or a remote paginated API. Clients shouldn't need to know — they just want to traverse it. Iterator exposes a uniform `hasNext()/next()` (or `__iter__/__next__`) cursor, decoupling traversal from storage and allowing multiple independent traversals at once.

Structure:

```

1 Aggregate { createIterator(): Iterator }
2 Iterator { hasNext(): bool; next(): T }
3     ConcreteIterator holds a cursor into ConcreteAggregate's internals.

```

```

1 class BinaryTree:
2     def __init__(self, value, left=None, right=None):
3         self.value, self.left, self.right = value, left, right
4     def __iter__(self):          # in-order traversal, hides tree structure
5         if self.left: yield from self.left
6         yield self.value
7         if self.right: yield from self.right
8
9 tree = BinaryTree(2, BinaryTree(1), BinaryTree(3))
10 for v in tree:                 # client traverses uniformly, knows nothing about nodes
11     print(v)                   # 1, 2, 3

```

```

1 // Java: implement Iterable<T> → works with for-each and streams
2 class Bag<T> implements Iterable<T> {
3     private final List<T> items = new ArrayList<>();
4     public void add(T t){ items.add(t); }
5     public Iterator<T> iterator(){ return items.iterator(); } // delegate or custom
6 }

```

When to use: Any custom collection you want traversable uniformly (for-each, streams); when you want multiple simultaneous traversals; lazy/streaming traversal (generators, paginated APIs). **When NOT to use:** Built-in collections already provide it; trivial fixed structures where direct access is clearer.

Real framework example: java.util.Iterator / Iterable (for-each loop), Python's iterator protocol (__iter__/_next__, generators), C# IEnumerable, database cursors, paginated REST clients exposing an iterator.

Trade-offs: Hides internal structure, uniform traversal, multiple concurrent iterators, supports lazy/infinite sequences. Overkill for simple arrays; **fail-fast vs. fail-safe** concerns (modifying a collection mid-iteration → ConcurrentModificationException in Java, or stale snapshots); stateful iterators complicate concurrency.

7.7 Chain of Responsibility

Intent: Avoid coupling the sender of a request to its receiver by giving **more than one object a chance to handle** the request. Chain the receivers and pass the request along until one handles it.

Problem it solves: An incoming HTTP request must pass through authentication, rate-limiting, validation, and logging. Hard-coding if (!auth) ... if (!rateOk) ... if (!valid) ... in one place is rigid and unordered. Chain of Responsibility makes each concern a handler linked to the next; each decides to handle and/or forward. You can reorder, insert, or remove stages without touching the others.

Structure:

```

Request → HandlerA → HandlerB → HandlerC → (end)
      |
      | each: handle it, and/or pass to next
Handler { setNext(h); handle(req){ ...; if (next) next.handle(req); } }

```

```

1 from abc import ABC, abstractmethod
2
3 class Handler(ABC):
4     def __init__(self): self._next = None
5     def set_next(self, h):
6         self._next = h; return h          # enables fluent chaining
7     @abstractmethod
8     def handle(self, req): ...
9     def _forward(self, req):
10         if self._next: self._next.handle(req)
11
12 class AuthHandler(Handler):
13     def handle(self, req):
14         if not req.get("user"): return print("REJECT: no auth")
15         print("auth ok"); self._forward(req)
16
17 class RateLimitHandler(Handler):
18     def handle(self, req):

```

```

19     if req.get("count", 0) > 100: return print("REJECT: rate limited")
20     print("rate ok"); self._forward(req)
21
22 class ValidateHandler(Handler):
23     def handle(self, req):
24         if not req.get("body"): return print("REJECT: empty body")
25         print("valid - processing!")
26
27 auth = AuthHandler()
28 auth.set_next(RateLimitHandler()).set_next(ValidateHandler()) # build chain
29 auth.handle({"user": "alice", "count": 5, "body": "data"}) # passes all stages

```

When to use: Request pipelines / middleware (web frameworks, interceptors), event bubbling, multi-level approval flows (expense → manager → director → VP), exception handling hierarchies. **When NOT to use:** When exactly one known handler exists (just call it); when a request *must* be handled (a chain can fall off the end unhandled — guard for that).

Real framework example: Servlet Filter chains, Spring Security filter chain, Java logging handler levels, Netty pipeline, Express/Django middleware, AWS API Gateway request flow.

Trade-offs: Decouples sender from receivers, reorder/add/remove handlers freely (OCP/SRP), each handler is focused. A request can go unhandled (silent failure), harder to debug ("which handler swallowed it?"), no guarantee of handling, runtime configuration can obscure flow.

7.8 Mediator

Intent: Define an object that encapsulates how a set of objects interact. Mediator promotes loose coupling by keeping objects from referring to each other explicitly, centralizing their many-to-many communication.

Problem it solves: In a dialog, toggling a checkbox enables a textbox, which enables a button... If each widget references every other widget directly, you get an n^2 tangle where every widget knows every other. A Mediator becomes the single hub: widgets notify the mediator, and the mediator orchestrates everyone else. The n^2 mesh becomes a star.

Structure:

Before (n^2): A ↔ B ↔ C ↓ X ↓ D ↔ E (everyone knows everyone)	After (Mediator): A → Mediator ← B ↑ C, D, E (everyone knows only the Mediator)
---	--

```

1 class DialogMediator:
2     def __init__(self):
3         self.checkbox = None; self.textbox = None; self.button = None
4     def changed(self, sender): # central orchestration
5         if sender is self.checkbox:
6             self.textbox.enabled = self.checkbox.checked
7             self.button.enabled = self.checkbox.checked
8
9 class Widget:
10     def __init__(self, mediator): self.mediator = mediator; self.enabled = False
11
12 class Checkbox(Widget):

```

```

13     def __init__(self, m): super().__init__(m); self.checked = False
14     def toggle(self):
15         self.checked = not self.checked
16         self.mediator.changed(self)          # tell the hub, not the peers
17
18 m = DialogMediator()
19 m.checkbox = Checkbox(m); m.textbox = Widget(m); m.button = Widget(m)
20 m.checkbox.toggle()          # mediator enables textbox + button

```

When to use: Complex many-to-many interactions among components you want decoupled (UI dialogs, air-traffic control, chat rooms routing messages between users). **When NOT to use:** Simple interactions (the mediator adds a needless hub); risk the mediator becomes a **God object** absorbing all logic.

Real framework example: `java.util.concurrent.ExecutorService` (mediates tasks↔threads), MVC controllers, message brokers, JMS, chat-server routing, Spring's event publisher as a lightweight mediator.

Trade-offs: Reduces coupling ($n^2 \rightarrow n$), centralizes interaction logic, components reusable in isolation. The mediator can grow into a God object / single point of complexity; centralization can become a bottleneck.

Mediator vs. Observer: Observer is one-to-many broadcast (subject doesn't coordinate observers); Mediator is many-to-many coordination (the hub actively orchestrates who does what). They're often combined (mediator uses observer to listen to components).

7.9 Memento

Intent: Without violating encapsulation, capture and externalize an object's internal state so the object can be **restored** to this state later (undo / snapshots / checkpoints).

Problem it solves: You want undo, but the editor's internal state is private — exposing it to the undo-manager would break encapsulation. Memento lets the object produce an opaque snapshot (the memento) that only it can read back, while a caretaker just stores snapshots without peeking inside.

Structure:

Originator (editor) — `save()` —> Memento (opaque state snapshot)
 Originator — `restore(memento)` —> back to that state
 Caretaker (history) stores mementos but CANNOT read their contents.

```

1 class EditorMemento:                                # opaque snapshot
2     def __init__(self, state): self._state = state  # "private" to originator
3     def _get(self): return self._state
4
5 class Editor:                                       # ORIGINATOR
6     def __init__(self): self.content = ""
7     def type(self, s): self.content += s
8     def save(self): return EditorMemento(self.content)    # create snapshot
9     def restore(self, m): self.content = m._get()          # restore snapshot
10
11 class History:                                     # CARETAKER — stores, never inspects
12     def __init__(self): self._mementos = []
13     def push(self, m): self._mementos.append(m)
14     def pop(self): return self._mementos.pop()

```

```

15
16 editor = Editor(); history = History()
17 editor.type("hello"); history.push(editor.save()) # checkpoint
18 editor.type(" world")
19 editor.restore(history.pop()) # undo → "hello"

```

When to use: Undo/redo, checkpoints, snapshots, transactional rollback, save-game systems — anywhere you must restore prior state without exposing internals. **When NOT to use:** When state is huge (snapshots cost memory — consider command-based undo or incremental deltas instead); when state is trivially public.

Real framework example: `java.io.Serializable` snapshots, database savepoints/rollback, editor undo stacks, Redux time-travel debugging, VM snapshots, `git stash`.

Trade-offs: Preserves encapsulation, clean undo/snapshot, caretaker stays dumb. Memory cost for many/large snapshots, lifecycle management of mementos, deep-copy concerns. **Command + Memento** is a common undo combo (command captures a memento before executing).

7.10 Visitor

Intent: Represent an operation to be performed on the elements of an object structure. Visitor lets you define a **new operation** without changing the classes of the elements on which it operates.

Problem it solves: You have a stable class hierarchy (AST nodes: `NumberNode`, `AddNode`, `MulNode`) and you keep needing *new operations* over it (evaluate, pretty-print, type-check, optimize). Adding each operation as a method on every node class bloats them and edits the whole hierarchy each time. Visitor moves operations *out* into visitor classes; each node just `accept(visitor)`s and calls back `visitor.visitX(this)` (**double dispatch**).

Structure:

```

Element { accept(v): v.visitThis(this) }      Visitor { visitNumber(n); visitAdd(a) }
  ▲                                           ▲
NumberNode AddNode MulNode          EvalVisitor PrintVisitor TypeVisitor
  (accept calls the right visit method - DOUBLE DISPATCH)

```

```

1 from abc import ABC, abstractmethod
2
3 class Node(ABC):
4     @abstractmethod
5     def accept(self, visitor): ...
6
7 class Number(Node):
8     def __init__(self, val): self.val = val
9     def accept(self, v): return v.visit_number(self) # double dispatch
10 class Add(Node):
11     def __init__(self, l, r): self.l, self.r = l, r
12     def accept(self, v): return v.visit_add(self)
13
14 class Visitor(ABC):
15     @abstractmethod
16     def visit_number(self, n): ...
17     @abstractmethod
18     def visit_add(self, a): ...
19

```

```

20 class EvalVisitor(Visitor):                # NEW operation, no Node edits
21     def visit_number(self, n): return n.val
22     def visit_add(self, a): return a.l.accept(self) + a.r.accept(self)
23
24 class PrintVisitor(Visitor):                # ANOTHER new operation
25     def visit_number(self, n): return str(n.val)
26     def visit_add(self, a): return f"({a.l.accept(self)} + {a.r.accept(self)})"
27
28 tree = Add(Number(3), Add(Number(4), Number(5)))
29 print(tree.accept(EvalVisitor()))          # 12
30 print(tree.accept(PrintVisitor()))         # (3 + (4 + 5))

```

When to use: A *stable* element hierarchy plus *many evolving operations* over it (compilers/AST, document object models, reporting over a fixed domain model). **When NOT to use:** When the *element hierarchy* changes often — adding a new element type forces editing *every* visitor (the dual of its strength). This is the famous **expression problem** trade-off.

Real framework example: Java annotation processing (javax.lang.model.element.ElementVisitor), the JDK FileVisitor (Files.walkFileTree), compiler/AST traversals (ANTLR visitors), NodeVisitor in Python's ast module, ASM bytecode visitors.

Trade-offs: Add operations without touching elements (OCP for operations), gather related logic in one visitor, accumulate state across a traversal. Adding an *element* type breaks all visitors, double-dispatch boilerplate, visitors may need access to element internals (breaks encapsulation), hard to read for newcomers.

7.11 Interpreter

Intent: Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language.

Problem it solves: You have a small, well-defined language to evaluate repeatedly — a boolean rule engine, a search filter syntax, simple arithmetic, regex-like matching. Interpreter models each grammar rule as a class; an expression tree of these objects evaluates a sentence via recursion.

Structure:

```

«interface» Expression { interpret(context): result }
    ▲
TerminalExpr (Number, Variable)      NonTerminalExpr (Add, Or, And — hold sub-Expressions)
    interpret() returns value          interpret() combines children's interpret()

```

```

1 from abc import ABC, abstractmethod
2
3 class Expr(ABC):
4     @abstractmethod
5     def interpret(self, ctx: dict): ...
6
7 class Var(Expr):                # terminal
8     def __init__(self, name): self.name = name
9     def interpret(self, ctx): return ctx[self.name]
10
11 class And(Expr):                # non-terminal
12     def __init__(self, l, r): self.l, self.r = l, r
13     def interpret(self, ctx): return self.l.interpret(ctx) and self.r.interpret(ctx)
14

```



```

15 class Or(Expr):
16     def __init__(self, l, r): self.l, self.r = l, r
17     def interpret(self, ctx): return self.l.interpret(ctx) or self.r.interpret(ctx)
18
19 # Rule: "premium AND (verified OR trusted)"
20 rule = And(Var("premium"), Or(Var("verified"), Var("trusted")))
21 print(rule.interpret({"premium": True, "verified": False, "trusted": True})) # True

```

When to use: A *simple, stable* grammar evaluated often (rule engines, filters, small DSLs, arithmetic/boolean expressions). **When NOT to use:** Complex or evolving grammars — hand-rolling each rule as a class becomes unmanageable; use a real parser generator (ANTLR, yacc) instead. It's the least-used GoF pattern in practice.

Real framework example: `java.util.regex.Pattern` (a compiled regex is an interpreter), `java.text.Format`, Spring Expression Language (SpEL), rule engines (Drools), database query planners conceptually.

Trade-offs: Easy to extend the grammar (new rule = new class), maps grammar directly to classes. Scales terribly for complex grammars (class per rule), performance overhead vs. a real parser, often the wrong tool — reach for a parser library.

7.12 Behavioral Patterns --- Comparison

Pattern	Intent	Smell it replaces
Strategy	Swap interchangeable algorithms	if type = ... algorithm selection
Observer	Notify many dependents on change	Polling / tight producer-consumer coupling
Command	Request as an object (undo/queue/log)	No undo, can't queue/replay actions
State	Behavior changes with internal state	Giant switch(status) in every method
Template Method	Fixed skeleton, overridable steps	Duplicated algorithm with small diffs
Iterator	Sequential access, hide structure	Exposing collection internals
Chain of Responsibility	Pipeline of handlers	Nested if deciding who handles
Mediator	Centralize many-to-many comms	n ² object interconnections
Memento	Snapshot/restore state	Exposing internals for undo
Visitor	New ops without editing elements	Methods bloating a stable hierarchy
Interpreter	Evaluate sentences of a grammar	Ad-hoc parsing of a small language

8 Modern / Architectural Patterns

The GoF book predates the web, dependency-injection containers, and reactive UIs. These patterns are everyday currency in modern codebases and appear constantly in interviews.

8.1 Dependency Injection / Inversion of Control (IoC)

Intent: Instead of an object creating its own dependencies (`this.db = new MySQLdb()`), the dependencies are **supplied** ("injected") from outside — usually by a container — so the object depends only on abstractions.

Problem it solves: Hard-coded new ConcreteClass() calls weld a class to its collaborators: you can't substitute a mock in tests, can't swap implementations, and creation logic spreads everywhere. DI inverts control of construction: a container (or the caller) builds the graph and hands each object what it needs.

```

1 // WITHOUT DI - OrderService is welded to MySQLRepo; untestable, inflexible
2 class OrderService {
3     private final Repo repo = new MySQLRepo(); // hard dependency
4 }
5
6 // WITH DI (constructor injection - the preferred form)
7 class OrderService {
8     private final Repo repo;
9     OrderService(Repo repo) { this.repo = repo; } // depends on the ABSTRACTION
10 }
11 // Production: new OrderService(new MySQLRepo());
12 // Test:      new OrderService(new InMemoryRepo()); // trivial to mock

```

Three injection styles: constructor (preferred — guarantees fully-initialized, immutable, required deps explicit), **setter** (optional deps, mutable), **field** (@Autowired on a field — concise but hides deps and hurts testability). **When NOT to use:** tiny scripts where a container is overkill; over-injecting (5+ constructor args signals an SRP violation).

Real framework example: Spring (@Component/@Autowired, the ApplicationContext is the IoC container), Google Guice, Dagger (compile-time DI), Angular's injector, .NET IServiceCollection, Python's dependency-injector/FastAPI Depends.

Trade-offs: Testability (inject mocks), loose coupling, centralized wiring, swap implementations by config. Indirection/"magic" (where did this object come from?), container learning curve, runtime wiring errors, can be overused. **DI is how you get Singleton's "one instance" without Singleton's global-state downsides** — the container owns one bean and injects it.

8.2 MVC / MVP / MVVM

Intent: Separate the **data (Model)**, the **UI (View)**, and the **glue logic** so each evolves independently and the model has no UI knowledge.

	MVC	MVP	MVVM
Glue	Controller	Presenter	ViewModel
View knows	Controller + Model	Presenter only	ViewModel (via binding)
Binding	Manual	Manual (presenter pushes to view)	Data binding (automatic)
View passivity	Active (observes model)	Passive (dumb)	Reactive (binds)
Typical use	Web (Rails, Spring MVC)	Android (older), WinForms	Android Jetpack, WPF, Vue/Knockout

MVC flow:

```

User → View → Controller → updates Model → Model notifies View (Observer) → View re-renders
                                   (Controller = Strategy for handling input)

```

Why it matters: MVC is itself a *composition of patterns* — the Model↔View link is **Observer**, the Controller's input handling is **Strategy**, and a View composed of nested widgets is **Composite**. This is the canonical "patterns combine" example. **Trade-offs:** separation of concerns,

testable model, parallel work. boilerplate, "Massive View Controller" anti-pattern when the controller absorbs everything (SRP violation).

8.3 Repository

Intent: Mediate between the domain and data-mapping layers, presenting an **in-memory collection-like interface** to domain objects (save, findById, findByEmail) while hiding the persistence mechanism.

Problem it solves: Scattering raw SQL / ORM calls through business logic couples your domain to the database, makes testing require a real DB, and means swapping Postgres→Mongo touches everything. A Repository abstracts persistence behind a collection-like interface; business code depends on the interface, not the data store.

```
1 from abc import ABC, abstractmethod
2
3 class UserRepository(ABC):
4     @abstractmethod
5     def find_by_id(self, id): ...
6     @abstractmethod
7     def save(self, user): ...
8
9 class SqlUserRepository(UserRepository):
10     def find_by_id(self, id): ...
11     def save(self, user): ...
12
13 class InMemoryUserRepository(UserRepository):
14     def __init__(self): self._store = {}
15     def find_by_id(self, id): return self._store.get(id)
16     def save(self, user): self._store[user.id] = user
```

When to use: Domain-driven designs, when you want testable business logic and the freedom to swap data stores. **When NOT to use:** Trivial CRUD apps where the ORM already gives a clean enough abstraction (a repository over a repository is ceremony). **Real framework example:** Spring Data JpaRepository, .NET EF repositories, Django's manager/queryset layer, Ruby's ActiveRecord (Active Record pattern — a *different* trade-off where the model knows how to persist itself). **Trade-offs:** testable, swappable persistence, centralizes queries. extra layer, can leak abstractions (generic repos that just wrap the ORM add little).

8.4 Null Object

Intent: Provide an object with **neutral ("do nothing") behavior** to represent the absence of a value — eliminating null checks.

Problem it solves: Code littered with `if (logger != null) logger.log(...)`. A Null Object is a real object implementing the same interface but doing nothing (or returning a safe default), so callers never special-case absence.

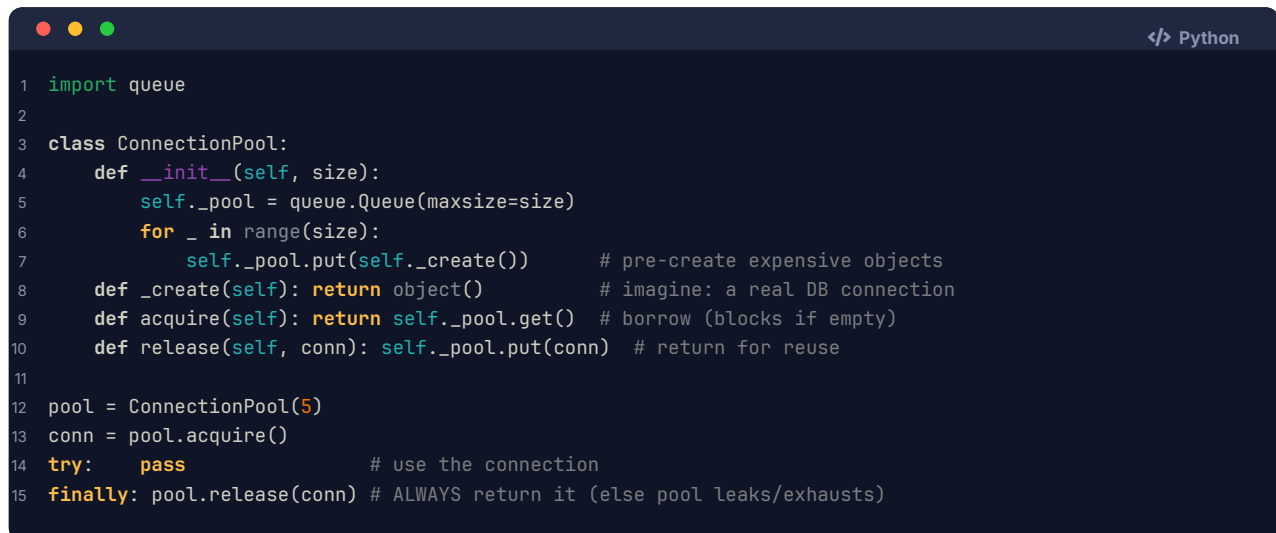
```
1 interface Logger { void log(String msg); }
2 class RealLogger implements Logger { public void log(String m){ System.out.println(m); } }
3 class NullLogger implements Logger { public void log(String m){ /* do nothing */ } }
4
5 // Caller never checks for null:
6 Logger logger = config.isLoggingOn() ? new RealLogger() : new NullLogger();
7 logger.log("hello"); // safe whether logging is on or off
```

When to use: A sensible "do nothing" default exists and you want to kill null checks (loggers, no-op listeners, empty collections). **When NOT to use:** When absence is genuinely an error that must surface (silently doing nothing can hide bugs). **Real framework example:** `Collections.emptyList()`, `Optional.empty()` (a richer cousin), SLF4J `NOPLogger`, Spring's no-op beans. **Trade-offs:** removes null checks (LSP-friendly), simpler call sites. can mask errors (a silent no-op when something was expected to happen).

8.5 Object Pool

Intent: Reuse a fixed set of **expensive-to-create objects** by borrowing from and returning to a pool, rather than constructing/destroying them repeatedly.

Problem it solves: Opening a DB connection or spinning a thread is expensive (TCP handshake, auth, OS thread setup). Creating one per request kills performance. An Object Pool pre-creates a bounded set, hands them out, and recycles them on return.



```

1 import queue
2
3 class ConnectionPool:
4     def __init__(self, size):
5         self._pool = queue.Queue(maxsize=size)
6         for _ in range(size):
7             self._pool.put(self._create()) # pre-create expensive objects
8     def _create(self): return object()      # imagine: a real DB connection
9     def acquire(self): return self._pool.get() # borrow (blocks if empty)
10    def release(self, conn): self._pool.put(conn) # return for reuse
11
12 pool = ConnectionPool(5)
13 conn = pool.acquire()
14 try:    pass # use the connection
15 finally: pool.release(conn) # ALWAYS return it (else pool leaks/exhausts)
  
```

When to use: Expensive, reusable, interchangeable resources under high churn (DB connections, threads, sockets, large buffers, game objects). **When NOT to use:** Cheap objects (pooling adds overhead and bugs), or when objects hold state that must be reset (forgetting to reset leaks state between borrowers). **Real framework example:** HikariCP / JDBC connection pools, `java.util.concurrent.ThreadPoolExecutor`, Apache Commons Pool, Netty buffer pools, game engine object pools. **Trade-offs:** amortizes creation cost, bounds resource usage, predictable performance. complexity (acquire/release discipline — leaks if not returned), must reset state between uses, sizing is tricky (too small = contention, too large = waste), thread-safety required.

9 Which Pattern Do I Use? (Decision Guide)

Map the *symptom* in the requirement to the pattern. In an interview, say the trigger phrase out loud.

The requirement says...	Reach for	Why
"There must be exactly one X"	Singleton (or DI-managed single bean)	One instance, controlled access
"Create an object but the type depends on input/subclass"	Factory Method	Defer instantiation

The requirement says...	Reach for	Why
"Create a <i>family</i> of matching objects"	Abstract Factory	Guarantee compatible products
"Many optional fields / immutable construction"	Builder	Avoid telescoping ctors
"Cloning is cheaper than rebuilding"	Prototype	Copy an exemplar
"Make this incompatible API fit ours"	Adapter	Interface conversion
"Add optional, stackable features at runtime"	Decorator	Compose responsibilities
"Hide this messy subsystem behind one call"	Facade	Simplify
"Control / defer / secure access to an object"	Proxy	Surrogate with control logic
"Treat single items and groups the same (tree)"	Composite	Part-whole hierarchy
"Two things vary independently (shape × renderer)"	Bridge	Avoid $n \times m$ explosion
"Millions of similar objects, memory-bound"	Flyweight	Share intrinsic state
"The algorithm must be swappable at runtime"	Strategy	Interchangeable algorithms
"Notify many things when one changes"	Observer	Pub/sub
"Need undo/redo, queue, or log of actions"	Command	Request as object
"Behavior depends on a status that transitions"	State	State machine
"Same flow, a few steps differ"	Template Method	Skeleton + hooks
"Traverse a collection without exposing it"	Iterator	Uniform cursor
"A request passes through stages until handled"	Chain of Responsibility	Pipeline/middleware
"Many components must coordinate (n^2)"	Mediator	Central hub
"Snapshot and restore state (undo)"	Memento	Encapsulated snapshot
"Add operations to a stable hierarchy"	Visitor	Externalize operations
"Evaluate a small custom grammar"	Interpreter	Grammar as classes
"Don't create my own dependencies"	Dependency Injection	Inject abstractions
"Separate data, UI, glue"	MVC/MVP/MVVM	Layered UI
"Abstract the data store"	Repository	Collection-like persistence
"Stop checking for null everywhere"	Null Object	Neutral default
"Reuse expensive objects"	Object Pool	Borrow/return

Default to the smallest tool. A plain function, polymorphism, or a first-class function beats a named pattern when the friction isn't real (KISS/YAGNI). Escalate to a pattern only when you *feel* the specific pain it relieves.

10 Patterns in the Wild & How They Combine

Real systems are **layered compositions** of patterns, not single ones. Recognizing the combinations is a senior signal.

MVC = Observer + Strategy + Composite. The View observes the Model (Observer); the Controller plugs in input-handling behavior (Strategy); a View built of nested widgets is a tree (Composite). The classic "patterns collaborate" example.

Spring Framework = Factory + Singleton + Proxy + Template Method. The BeanFactory/ApplicationContext is a Factory producing beans; beans are singleton-scoped by default (DI-managed Singleton); @Transactional/AOP wraps beans in dynamic **Proxies**; JdbcTemplate/RestTemplate are Template Methods (fixed flow, your callbacks fill steps).

Java I/O = Decorator + Adapter + Iterator. BufferedReader(new InputStreamReader(new FileInputStream(f))) stacks **Decorators**; InputStreamReader is an **Adapter** (bytes→chars); collections traverse via **Iterator**.

React = Observer + Composite + Decorator/Strategy. Components observe state and re-render (Observer); the component tree is a **Composite**; Higher-Order Components and hooks wrap behavior (Decorator-ish); render props pass behavior in (Strategy-ish).

A plugin system = Factory + Strategy + Observer. A Factory instantiates plugins by name; each plugin is a Strategy implementing a common interface; the host emits lifecycle events plugins observe.

Undo system = Command + Memento. Each Command captures a Memento (state snapshot) before executing; the history replays/reverts them.

Compiler = Visitor + Interpreter + Composite + Factory. The AST is a Composite tree; passes (type-check, optimize, codegen) are Visitors; a small expression evaluator is an Interpreter; node creation uses Factories.

A web request pipeline = Chain of Responsibility + Strategy + Decorator. Middleware is a Chain; each handler may select behavior via Strategy; route handlers can be wrapped (Decorator) with logging/auth.

Interview gold: When asked "how do patterns combine?", give MVC = Observer + Strategy + Composite, or Spring = Factory + Singleton + Proxy. It shows you understand patterns as a *system*, not flashcards.

11 Anti-Patterns

An anti-pattern is a *common but counterproductive* solution. Recognizing them — and naming the refactor — is as valuable as knowing the patterns.

11.1 God Object (a.k.a. Blob)

What: One class that knows/does everything (OrderManager with 4,000 lines handling pricing, payment, email, persistence). **Why bad:** violates SRP, untestable, every change risks breaking unrelated features, merge-conflict magnet. **Fix:** Extract responsibilities into focused collaborators (SRP); use Facade if you still want a simple entry point; push behavior into the right domain objects (kill the anemic model).

11.2 Golden Hammer

What: "When all you have is a hammer, everything looks like a nail" — applying one familiar pattern/tool everywhere (Singleton for every service, inheritance for everything, microservices for a CRUD app). **Why bad:** forces ill-fitting solutions, adds complexity. **Fix:** match the tool to the problem; keep a broad toolkit; justify each choice by the *force* it relieves.

11.3 Spaghetti Code

What: Tangled control flow with no clear structure — deep nesting, goto-like jumps, hidden global state, no layering. **Why bad:** unreadable, unmaintainable, fear-driven changes. **Fix:** introduce layers/modules, extract methods, apply SRP, replace conditionals with polymorphism (Strategy/State), establish clear boundaries.

11.4 Poltergeist (a.k.a. Gypsy class)

What: A short-lived, do-nothing class that just passes control to another (a "manager" that only forwards calls). It haunts the design — appears, does little, vanishes. **Why bad:** needless indirection, clutter, obscures real flow. **Fix:** remove it; call the real worker directly; merge its (trivial) logic into a meaningful class.

11.5 Premature Abstraction / Speculative Generality

What: Building interfaces, generics, and extension points for variation that *might* happen but never does (AbstractFactoryProviderStrategyFactory for a feature with one implementation). **Why bad:** YAGNI violation — pays complexity now for flexibility never used; harder to read and change. **Fix:** **Rule of Three** — introduce the abstraction on the *third* time you actually need to vary something, not the first. Start concrete, refactor to abstract when a real second case arrives.

11.6 Singleton Abuse

What: Reaching for Singleton as a glorified global variable for state that isn't truly single (or just to avoid passing arguments). **Why bad:** hidden dependencies, global mutable state, untestable (can't mock), thread-safety hazards, hidden coupling. **Fix:** use **Dependency Injection** of a single container-managed instance — you keep "one instance" but make it explicit, mockable, and swappable.

11.7 Bonus anti-patterns worth naming

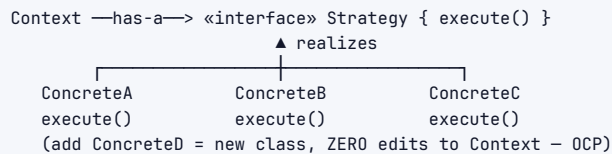
- › **Anemic Domain Model:** classes with only getters/setters, all logic in "Service" classes — not OOP. *Fix:* put behavior on the object that owns the data.
- › **Lava Flow:** dead code no one dares delete. *Fix:* test coverage + delete.
- › **Boat Anchor:** keeping unused components "just in case." *Fix:* delete; version control remembers.
- › **Yo-Yo Problem:** inheritance so deep you bounce up and down the hierarchy to follow logic. *Fix:* flatten; favor composition.

Anti-pattern	Core problem	The fix
God Object	One class does everything	SRP — extract collaborators
Golden Hammer	One pattern for every problem	Match tool to problem
Spaghetti	No structure/layering	Layer, extract, polymorphism
Poltergeist	Useless pass-through class	Delete, call worker directly
Premature Abstraction	Flexibility never used	Rule of Three; start concrete

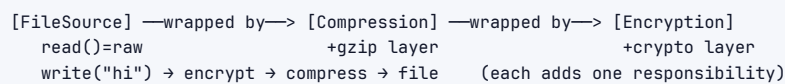
Anti-pattern	Core problem	The fix
Singleton Abuse	Global mutable state	Dependency Injection

12 Architecture / Diagrams

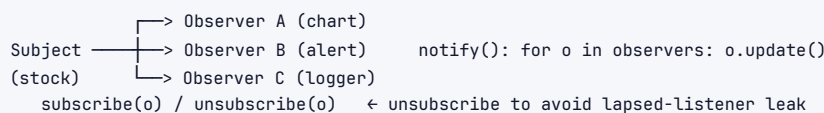
Strategy (composition + OCP):



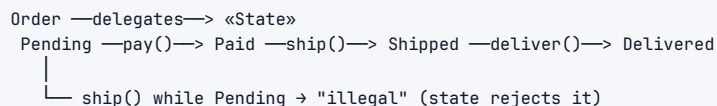
Decorator stacking (composition over inheritance):



Observer (one-to-many broadcast):



State (object swaps its own behavior):



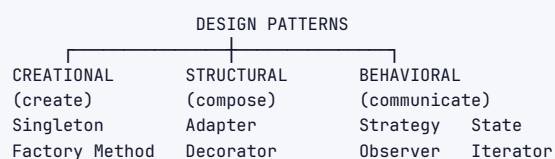
Composite (uniform tree):



Chain of Responsibility (pipeline):



Pattern family map:



Abstract Factory	Facade	Command	Chain of Resp.
Builder	Proxy	Template	Mediator
Prototype	Composite	Memento	Visitor
	Bridge	Interpreter	
	Flyweight		

13 Real-World Examples

13.1 Java Standard Library

- › **Decorator:** `BufferedReader`, `BufferedInputStream`, `GZIPInputStream`, `Collections.synchronizedList()`.
- › **Adapter:** `InputStreamReader` (bytes→chars), `Arrays.asList()`.
- › **Factory Method:** `Calendar.getInstance()`, `NumberFormat.getInstance()`, `Optional.of()`.
- › **Singleton:** `Runtime.getRuntime()`, enums.
- › **Strategy:** Comparator passed to `Collections.sort()`.
- › **Observer:** `PropertyChangeListener`, deprecated `java.util.Observer`.
- › **Template Method:** `AbstractList`, `HttpServlet.service()`.
- › **Iterator:** the entire `Iterable/Iterator` framework behind `for-each`.
- › **Proxy:** `java.lang.reflect.Proxy`, RMI stubs.
- › **Flyweight:** `Integer.valueOf()` cache, the String pool.

13.2 Spring Framework

`ApplicationContext` (Factory + IoC container), default singleton beans, `@Transactional/AOP` (Proxy), `JdbcTemplate/RestTemplate` (Template Method), `ApplicationEvent` (Observer), `@Configuration` (Factory + Builder-ish).

13.3 React / Frontend

Components observing state and re-rendering (Observer), the component tree (Composite), HOCs and hooks (Decorator), render props / dependency-passing (Strategy), Context (Mediator-ish), Redux store (Observer + Command — actions are commands, reducers interpret them).

13.4 Backend / Distributed (the distributed cousins)

- › **Strategy** → pluggable load-balancing / routing / retry policies.
- › **Observer** → Kafka/RabbitMQ pub-sub, webhooks.
- › **Command** → task queues (Celery, Sidekiq), CQRS commands, event sourcing.
- › **Chain of Responsibility** → API gateway / middleware filters (Spring Security, Express).
- › **Proxy** → service mesh sidecars (Envoy), API gateways, caching layers.
- › **Facade** → BFF (backend-for-frontend), SDK clients.
- › **Circuit Breaker / Saga / CQRS** → architectural-scale behavioral patterns (see §System Design).

14 Real-Life Analogies

One restaurant — every pattern is a role, a tool, or a workflow in the same kitchen.

Pattern	Analogy
Singleton	The restaurant's single head chef — there is exactly one, everyone routes orders through them
Factory Method	"Make me the soup of the day" — the kitchen decides which soup to cook based on the day
Abstract Factory	Ordering a "tasting menu" — every course (appetizer, main, dessert) comes from one matching themed set
Builder	Building a custom burger at the counter — bun, then patty, then toppings, each step optional, one "done" at the end
Prototype	The signature dish recipe card — clone it and tweak the spice rather than inventing from scratch
Adapter	A wine-bottle opener that fits both cork and screw-cap — bridges two incompatible bottle types
Decorator	Building a coffee order — espresso, then add milk, then add caramel; each layer adds without remaking the cup
Facade	The waiter — you say "I'll have the special," they coordinate kitchen, bar, and till; you never enter the kitchen
Proxy	The host at the door — screens reservations, checks the list, only seats you if you qualify
Composite	A banquet order — "table of 8" is treated like one order, yet each guest's plate is also handled individually
Bridge	Menu (dishes) × cooking method (grill/fry/steam) — add a dish or a method independently, no combinatorial menu
Flyweight	One shared salt shaker per table instead of a personal one per guest — shared, identical, reused
Strategy	Choosing a payment method at the till — cash, card, or app; same checkout, swappable method
Observer	The "order up!" bell — when a dish is ready, every relevant waiter is notified at once
Command	A written order ticket — queued on the rail, can be re-fired, and "void" undoes it
State	A table's lifecycle — Empty → Seated → Ordered → Served → Paying; each state allows different actions
Template Method	The standard service script — greet, take order, serve, bill; the steps are fixed, the dishes vary
Iterator	The waiter going table to table in order, taking each order without a floor map
Chain of Responsibility	A complaint escalating — waiter → manager → owner, until someone resolves it
Mediator	The expediter ("pass") who coordinates between cooks, waiters, and bar so they don't all shout at each other
Memento	Saving a half-eaten meal as a "to-go box" to resume the exact state later
Visitor	The health inspector who walks the same kitchen applying a new checklist without changing the kitchen
Interpreter	Reading a recipe's shorthand ("2T butter, fold, rest") — each token interpreted into an action
Dependency Injection	The supplier delivers ingredients to the kitchen — chefs don't go forage; they're handed what they need
Object Pool	A rack of clean pans reused across orders rather than buying a new pan per dish

15 Memory Tricks / Mnemonics

15.1 The three families

"Create, Structure, Behave" — Creational (how objects are *created*), Structural (how they're *composed*), Behavioral (how they *communicate*).

15.2 Creational --- "SABFP"

Singleton · Abstract Factory · Builder · Factory Method · Prototype → *"So A Builder Factory Prototypes."*

15.3 Structural --- "A Big Cat Drinks From Pool" (ABCDFP + Proxy)

Adapter · Bridge · Composite · Decorator · Facade · flyProxy/Flyweight.

15.4 Behavioral --- "Smart Os Cause State To Iterate Calmly, Mostly Visiting Interpreters"

Strategy · Observer · Command · State · Template Method · Iterator · Chain of Resp. · Mediator · Memento · Visitor · Interpreter.

15.5 Killer distinctions (the ones interviewers probe)

- **Strategy vs. State:** *Strategy* — **you** pick the algorithm (it doesn't change itself). *State* — the object **changes its own** behavior as it transitions.
- **Decorator vs. Proxy:** same structure (both wrap). *Decorator* adds responsibilities (stackable); *Proxy* controls access (usually one).
- **Adapter vs. Facade:** *Adapter* makes one interface *match* an expected one. *Facade* makes many interfaces *simpler*.
- **Factory Method vs. Abstract Factory:** *Method* makes **one** product (inheritance). *Abstract* makes a **family** (composition).
- **Decorator vs. Inheritance:** decorate at **runtime** (compose); inherit at **compile time** (fixed).
- **Template Method vs. Strategy:** *Template* varies **steps** via inheritance; *Strategy* swaps the **whole algorithm** via composition.
- **Mediator vs. Observer:** *Observer* = broadcast (one→many); *Mediator* = coordination hub (many↔many).

16 Common Interview Questions

16.1 Q1: What is a design pattern, and when should you NOT use one?

Model answer: A design pattern is a named, reusable *template* for solving a recurring object-oriented design problem — it captures an intent, a structure (the collaborating classes), and trade-offs. Patterns give a shared vocabulary and proven, SOLID-aligned structures that reduce coupling and ease change. You should NOT use one when it adds indirection without removing real pain — a premature pattern violates YAGNI/KISS, adds classes, and makes code harder to read. Reach for a pattern only when you *feel* the specific friction it relieves (a growing switch on type → Strategy or State; subclass explosion → Decorator). The senior skill is right-sizing: a plain function or polymorphism first, a named pattern only when justified.

Follow-ups:

- *"Give an example of over-engineering with patterns."* → Wrapping a single, never-varying implementation in a Strategy interface and Factory — you pay the indirection cost for flexibility you'll never use.

- › *"How do you decide?"* → Rule of Three: introduce the abstraction on the third time you actually need to vary something, not the first.

16.2 Q2: Explain Strategy vs. State --- they look identical.

Model answer: Structurally they're the same: a context object delegates to a swappable helper behind an interface. The **intent** differs. **Strategy** lets a *client* choose among interchangeable algorithms that don't know about each other and rarely change mid-flow — it answers *"how do I do this?"* (e.g., which sort, which payment method). **State** encapsulates behavior that depends on the object's internal status, and crucially the states *transition to one another* (a state often decides the next state) — it answers *"what can I do now?"* (e.g., an order in PENDING vs. SHIPPED). Strategy is set from outside; State drives itself through a lifecycle.

Follow-ups:

- › *"Which replaces a giant switch on a status field across many methods?"* → State.
- › *"Do strategies know each other?"* → No. States do (they reference successor states).
- › *"Can State be implemented with a transition table instead of classes?"* → Yes — a `Map<State, Map<Event, State>>`; cleaner when transitions dominate behavior.

16.3 Q3: Why is Singleton often called an anti-pattern, and what's the alternative?

Model answer: Singleton introduces hidden **global mutable state**: a method calling `Config.get()` doesn't declare that dependency in its signature, so coupling is invisible. It makes unit testing hard — you can't substitute a mock, and state leaks between tests. It adds thread-safety complexity and tends to proliferate ("everything's a singleton"). The alternative is **Dependency Injection**: have a container create *one* instance and inject it where needed. You keep the "exactly one instance" guarantee but make the dependency explicit, mockable, and swappable. Spring beans are exactly this — singleton-scoped but DI-managed.

Follow-ups:

- › *"Show a thread-safe Singleton in Java."* → Bill Pugh holder idiom (lazy + thread-safe + no sync cost) or an enum (also serialization/reflection-safe).
- › *"Why is volatile required in double-checked locking?"* → Without it, instruction reordering can publish a reference to a partially-constructed object to another thread.

16.4 Q4: Decorator vs. Proxy vs. Inheritance --- distinguish them.

Model answer: **Decorator** and **Proxy** both wrap an object behind the same interface. **Decorator** *adds responsibilities* and you can stack many in any order (Java I/O streams: buffering + compression + encryption). **Proxy** *controls access* to exactly one object — lazy creation (virtual), authorization (protection), remote calls (RPC stub), or caching — without changing what the object does. **Inheritance** adds behavior at compile time, can't be combined or removed at runtime, and leads to subclass explosion when features combine; Decorator composes instead. Rule of thumb: adding features → Decorator; gatekeeping access → Proxy; fixed specialization with a true IS-A → inheritance (sparingly).

Follow-ups:

- › *"Give the canonical Decorator in the JDK."* → `new BufferedReader(new InputStreamReader(new FileInputStream(f)))`.
- › *"Which does Spring @Transactional use?"* → Proxy (a dynamic proxy opens/commits the transaction around the bean method).

16.5 Q5: Factory Method vs. Abstract Factory?

Model answer: Factory Method is a single (often abstract) method that creates *one* product, where a *subclass* decides the concrete type — it uses inheritance. **Abstract Factory** is an *object* exposing several factory methods to create a *family* of related products that must be used together (a matching Button + Checkbox per OS), guaranteeing compatibility — it uses composition and is often *implemented with* Factory Methods. Trade-off note: Abstract Factory is flexible against new *families* but rigid against new *product kinds* (adding a slider edits every factory).

Follow-ups:

- › "When do you prefer a simple static factory over either?" → Small cases with a finite, stable set of types — one create(type) method is clearer than a class hierarchy.

16.6 Q6: How does Observer avoid tight coupling, and what's the classic bug?

Model answer: The subject knows only the Observer interface, not concrete observers; observers register/unregister at runtime. The subject broadcasts state changes without knowing who listens or what they do, so producers and consumers evolve independently — this is the basis of event systems and reactive UIs. Push vs. pull: push sends the changed data in the notification; pull sends "something changed" and lets the observer query the subject. The classic bug is the **lapsed-listener leak**: the subject holds strong references to observers, so an observer that's discarded without calling unsubscribe() is kept alive forever (memory leak) and keeps receiving useless notifications. Fixes: explicit unsubscribe, weak references, or lifecycle-scoped subscriptions.

Follow-ups:

- › "What about notification order and storms?" → Order is undefined — don't depend on it; cascading updates can storm, so batch/debounce.
- › "Thread safety?" → Modifying the observer list while iterating it is a race; use CopyOnWriteArrayList or snapshot before notifying.

16.7 Q7: Give a pattern that embodies the Open/Closed Principle and explain how.

Model answer: Strategy. Adding a new algorithm means writing a new class implementing the strategy interface; the context is *closed for modification* (untouched) yet the system is *open for extension*. It also embodies DIP (the context depends on the abstraction, not concrete strategies) and composition over inheritance. Factory Method and Decorator are equally valid answers — Factory lets you add product types without editing the creator's logic; Decorator adds responsibilities without modifying the component.

Follow-ups:

- › "What's the cost of OCP here?" → More classes and indirection; only worth it if the variation point will actually be varied (Rule of Three).

16.8 Q8: What pattern fits a request/middleware pipeline?

Model answer: Chain of Responsibility. Each handler (auth → rate-limit → validation → logging) gets a chance to process the request and decides whether to handle it and/or pass it to the next handler. It decouples the sender from the set of handlers, lets you reorder/insert/remove stages without touching others (OCP/SRP), and short-circuit (reject) early. The risk is a request falling off the end unhandled, so guard for that. This is exactly how Servlet filters, Spring Security, and Express/Django middleware work.

Follow-ups:

- › "How does it differ from Decorator?" → Decorator always delegates and *augments*; a CoR handler may *stop* the chain (handle-or-forward), and the focus is "who handles this," not "add behavior."

16.9 Q9: How would you implement undo/redo, and which patterns?

Model answer: Command + Memento. Each user action becomes a `Command` object with `execute()` and `undo()`. An invoker maintains a history stack of executed commands and a redo stack. For complex state where computing an inverse is hard, the command captures a **Memento** (an opaque snapshot of the relevant state) before executing and restores it on undo — this preserves encapsulation since only the originator can read the memento. Redo re-executes the popped command. Watch memory for long histories (snapshots are costly — prefer command inverses or deltas when possible).

Follow-ups:

- › *"Command vs. Memento for undo?"* → `Command` stores the *operation* (and how to reverse it); `Memento` stores the *state*. Use `Command` for invertible ops, `Memento` when reversal is impractical.
- › *"How do macros fit?"* → A composite command (list of commands) — itself an instance of `Composite` + `Command`.

16.10 Q10: When would you choose Composition over Inheritance, and which patterns reflect that?

Model answer: Prefer composition unless there's a genuine, stable IS-A relationship and you want the full parent API. Inheritance creates tight coupling (fragile base class — a parent change breaks children), fixes behavior at compile time, and explodes combinatorially when features combine. Composition delegates to a held object through its interface, is swappable at runtime, and keeps hierarchies flat. Most patterns are composition mechanisms: **Strategy** (hold an algorithm), **Decorator** (wrap a component), **Bridge** (abstraction holds an implementor), **State** (hold the current state), **Adapter/Proxy** (wrap an object). Inheritance shows up mainly in **Template Method** (override steps) and **Factory Method** (subclass picks the type).

Follow-ups:

- › *"When is inheritance the right call?"* → True IS-A, shallow hierarchy (≤ 2 levels), you want polymorphic dispatch and the full base contract (e.g., `ArrayList` extends `AbstractList`).

16.11 Q11: How do real frameworks combine multiple patterns? Give a concrete example.

Model answer: MVC = Observer + Strategy + Composite. The View observes the Model and re-renders on change (`Observer`); the Controller plugs in input-handling behavior (`Strategy`); a View built from nested widgets is a tree (`Composite`). Another: **Spring = Factory + Singleton + Proxy + Template Method** — the `ApplicationContext` is a bean Factory, beans are singleton-scoped (DI-managed Singleton), `@Transactional` wraps beans in dynamic Proxies, and `JdbcTemplate` is a Template Method. Recognizing these combinations shows you understand patterns as a collaborating system, not isolated flashcards.

Follow-ups:

- › *"How does Java I/O combine patterns?"* → `Decorator` (stream wrapping) + `Adapter` (`InputStreamReader` bytes→chars) + `Iterator` (collection traversal).

16.12 Q12: What's an anti-pattern, and how do you fix the God Object?

Model answer: An anti-pattern is a common but counterproductive "solution" that creates more problems than it solves. The **God Object** is a class that knows and does everything (an `OrderManager` handling pricing, payment, email, persistence) — it violates SRP, is untestable, and every change risks breaking unrelated features. The fix: extract each responsibility into a focused collaborator (SRP), push behavior into the right domain objects (kill the anemic model), and if you still want a single easy entry point, put a thin **Facade** in front of the collaborators —

the Facade simplifies access *without* absorbing the logic. Other anti-patterns to name: Golden Hammer (one pattern everywhere → match tool to problem), Premature Abstraction (Rule of Three), Singleton Abuse (use DI), Spaghetti (layer + polymorphism).

Follow-ups:

- › "How is a Facade different from a God Object?" → A Facade *delegates* to focused subsystem classes (thin coordinator); a God Object *contains* all the logic itself.

17 Senior-Level Discussion Points

- › **Patterns are trade-offs, not virtues.** Every pattern adds indirection and classes; you must justify the cost against the pain it relieves. The senior move is often *not* using a pattern — recognizing when a plain function, polymorphism, or a first-class function is enough.
- › **Language shapes patterns.** Many GoF patterns are workarounds for missing language features. First-class functions collapse Strategy/Command/Observer into "pass a function." Python's duck typing makes Adapter nearly free. Go favors composition + small interfaces and has no inheritance. In functional languages, immutability + higher-order functions replace much of the behavioral catalog. Naming this nuance signals depth.
- › **Encapsulate-what-varies is the meta-principle.** Almost every pattern is the same move: find the axis of change, hide it behind an interface, depend on the interface (OCP + DIP). If you internalize this, you can *derive* the right pattern rather than recall it.
- › **Patterns combine and layer.** Real systems are compositions (MVC, Spring, React, compilers). Discuss how they collaborate, not just individual definitions.
- › **Thread-safety in patterns is a frequent follow-up.** Singleton needs safe lazy init (volatile DCL, holder idiom, enum). Observer needs a thread-safe observer list (CopyOnWriteArrayList) and care iterating while modifying. Flyweight's shared intrinsic state *must* be immutable. Object Pool needs synchronized borrow/return.
- › **Anti-patterns are half the skill.** Recognizing God objects, golden hammers, premature abstraction, poltergeists, and singleton abuse — and naming the refactor — is as valuable as knowing the 23 patterns. Interviewers love "what's wrong with this design?"
- › **Architectural patterns are the distributed cousins.** CQRS, Saga, Circuit Breaker, Event Sourcing, Sidecar/Ambassador (a distributed Proxy) — these scale GoF intents to services. Mentioning the bridge from LLD patterns to HLD patterns shows range.
- › **The Rule of Three governs abstraction.** Don't abstract on the first occurrence (premature), or even the second. On the third real need to vary, introduce the abstraction. Speculative generality is a tax you pay forever for flexibility you may never use.

18 Typical Mistakes Candidates Make

Mistake	Why it's wrong	What to do instead
Pattern soup — forcing patterns everywhere	Over-engineering; indirection with no payoff	Smallest solution first; escalate only on real friction
Naming the structure, not the intent	Reciting UML without <i>why</i> sounds cargo-cult	Explain the force it relieves and the trade-off
Singleton everywhere	Global mutable state, untestable	Dependency Injection of a single managed instance
Confusing Strategy/State or Decorator/Proxy	Shows shallow understanding	Memorize the <i>intent</i> distinction, not just the shape

Mistake	Why it's wrong	What to do instead
Inheritance-based "decoration"	Subclass explosion	Compose decorators at runtime
Ignoring thread-safety	Broken Singleton/Observer/Flyweight under concurrency	Proactively address it (volatile, immutability, CopyOnWriteArrayList)
Forgetting the lapsed-listener leak	Observer memory leaks in long-running systems	Always provide/explain unsubscribe or weak refs
Premature abstraction	Interfaces for variation that never comes	Rule of Three — start concrete
Shallow vs. deep copy in Prototype	Cloned objects share mutable state	Choose deep copy deliberately; document it
Anemic model + God service	Logic dumped in services, not objects	Put behavior on the object that owns the data
Adding patterns without trade-off talk	Misses the senior signal	Always state the cost ("more classes, but OCP")

19 How This Connects to Other Topics

- › **OOP (§12) & LLD (§06)** — Patterns *are* applied OOP; LLD/machine-coding rounds expect you to reach for them appropriately. Patterns are the vocabulary; LLD is where you speak it.
- › **System Design / HLD (§05)** — Architectural patterns (CQRS, Saga, Circuit Breaker, Pub/Sub, Sidecar, API Gateway) are the *distributed* descendants of GoF intents: Pub/Sub is Observer at scale, a Sidecar is a Proxy, a Gateway is a Facade + Chain of Responsibility.
- › **Concurrency (§01)** — Singleton/Flyweight/shared state demand thread-safety; producer-consumer is structurally Command + a queue; Object Pool underlies thread pools and connection pools.
- › **Databases / ORM (§08)** — Repository abstracts the data store; Active Record (model persists itself) is the alternative trade-off; Unit of Work groups operations into a transaction; connection pooling is Object Pool.
- › **Testing** — Good pattern use enables testing: DI injects mocks, Strategy swaps test algorithms, Repository's in-memory impl avoids a real DB, Null Object simplifies test setup. Patterns and testability rise together.
- › **Frameworks you use daily** — Spring, React, the JDK, Django, Hibernate are *built from* these patterns; understanding them lets you read framework source and debug "magic."

20 FAANG Interview Tips

1. **Let the requirement reveal the pattern.** Say the trigger out loud: "the algorithm must be swappable" → Strategy; "notify many on change" → Observer; "undo" → Command + Memento; "behavior depends on a status that transitions" → State. Naming it as you go signals pattern literacy.
2. **Prefer the smallest solution.** A function, polymorphism, or a first-class function before a named pattern. Escalate only when the friction is real. Over-patterning is penalized as hard as under-designing.
3. **Always state the trade-off.** "Decorator avoids subclass explosion but produces many small wrapper objects and ordering can matter." The trade-off sentence is the senior signal.
4. **Know the 5 cold:** Strategy, Observer, Factory, Decorator, Singleton — they cover the majority

of LLD prompts. Then State, Command, Adapter, Composite, Builder for the next tier.

5. **Nail the killer distinctions:** Strategy vs. State, Decorator vs. Proxy, Adapter vs. Facade, Factory Method vs. Abstract Factory. Interviewers probe these to separate memorizers from understanders.
6. **Show the extensibility story.** After designing, say: "to add an EV charging spot, I subclass / add a new strategy — zero edits to existing code (OCP)." This is the #1 senior moment.
7. **Address concurrency proactively** when the system is multi-threaded — Singleton (volatile DCL / holder idiom / enum), Observer (CopyOnWriteArrayList, lapsed-listener leak), Flyweight (immutability).
8. **Recognize anti-patterns out loud** if you spot a God object / golden hammer / premature abstraction in a given design — and name the refactor. "What's wrong with this design?" is a common prompt.
9. **Demonstrate combination knowledge.** "MVC is Observer + Strategy + Composite; Spring is Factory + Singleton + Proxy." It shows systemic understanding.
10. **Amazon-specific:** map choices to Leadership Principles — "Dive Deep" (knowing pattern internals like why volatile is needed in DCL), "Invent and Simplify" (choosing the *right* pattern, not the most complex), "Insist on Highest Standards" (clean, SOLID code).

21 Revision Cheat Sheet

21.1 Families

Creational (how objects are *created*) · **Structural** (how they're *composed*) · **Behavioral** (how they *communicate*).

21.2 One-liners

Pattern	Family	One-liner
Singleton	C	One shared instance (use sparingly; prefer DI)
Factory Method	C	Subclass decides the concrete type (inheritance)
Abstract Factory	C	Create compatible product <i>families</i> (composition)
Builder	C	Step-by-step / immutable construction; no telescoping ctors
Prototype	C	Clone an exemplar instead of new
Adapter	S	Convert one interface to another (compatibility)
Decorator	S	Stack responsibilities at runtime (Java I/O)
Facade	S	Simple front for a complex subsystem
Proxy	S	Controlled stand-in (virtual/protection/remote/cache)
Composite	S	Treat leaf & group uniformly (trees)
Bridge	S	Split abstraction from impl (avoid $n \times m$)
Flyweight	S	Share intrinsic state to save memory
Strategy	B	Swap interchangeable algorithms (OCP + DIP)
Observer	B	Pub/sub; notify many on change (mind the leak)
Command	B	Request as object (undo/queue/log)
State	B	Behavior changes as state transitions (replaces switch)
Template Method	B	Fixed skeleton, overridable steps (Hollywood)
Iterator	B	Sequential access, hide structure
Chain of Responsibility	B	Pipeline of handlers (middleware)

Pattern	Family	One-liner
Mediator	B	Central hub for many-to-many comms
Memento	B	Snapshot/restore state (undo, encapsulated)
Visitor	B	Add operations to a stable hierarchy
Interpreter	B	Evaluate sentences of a small grammar
Dependency Injection	Modern	Inject abstractions; testable, swappable
MVC/MVP/MVVM	Modern	Separate data / UI / glue
Repository	Modern	Collection-like abstraction over persistence
Null Object	Modern	Neutral do-nothing default (kills null checks)
Object Pool	Modern	Borrow/return expensive reusable objects

21.3 Killer distinctions (memorize)

- › **Strategy vs. State:** you pick the algorithm | object changes itself.
- › **Decorator vs. Proxy:** adds behavior (stack) | controls access (one).
- › **Adapter vs. Facade:** make it *match* | make it *simpler*.
- › **Factory Method vs. Abstract Factory:** one product (inherit) | family (compose).
- › **Template Method vs. Strategy:** vary steps (inherit) | swap whole (compose).
- › **Mediator vs. Observer:** coordinate many↔many | broadcast one→many.

21.4 Singleton thread-safe forms (Java)

Eager (simple) · DCL with `volatile` (lazy) · **Bill Pugh holder** (best general) · **enum** (Bloch's choice — serialization/reflection-safe).

21.5 Anti-patterns → fixes

God Object → SRP/extract · Golden Hammer → match tool · Spaghetti → layer + polymorphism · Poltergeist → delete · Premature Abstraction → Rule of Three · Singleton Abuse → DI.

21.6 Golden rules

1. **Encapsulate what varies, program to interfaces, favor composition.**
2. **Don't pattern what a function can do** (KISS/YAGNI).
3. **Rule of Three** — abstract on the third real need, not the first.
4. **Name the pattern *and* the trade-off** — that's the senior signal.

Study tip: For each of the five core patterns (Strategy, Observer, Factory, Decorator, Singleton), be able to (1) state the intent in one line, (2) write the code from memory, (3) name a JDK/Spring/React example, and (4) recite the trade-off. Then drill the killer distinctions until they're reflexive — that's what separates "I memorized the catalog" from "I understand design."